State Space, Stability and the Acrobot

Lecture notes, starting from zero

Control Systems

How to use these notes

These notes assume no background beyond basic algebra. Everything else is explained when it is first needed: what a derivative means, what a matrix is, how to multiply a matrix by a vector, what a determinant is, and what complex numbers are. If you already know a topic, skip it. Nothing from earlier in the course is assumed; in particular transfer functions are introduced from scratch in Section 10.

The notes follow the slides. Part 1 builds up state space and stability; Part 2 applies it to the acrobot. The slide numbers for each section are on the right of its heading. Section 1 is a toolkit; read it first, or come back to it whenever a later section uses something you have not seen.

The main symbols are listed in the glossary at the end.

Contents

I State space and stability **3**

1 Toolkit **3**
- 1.1 Rates of change and the dot notation . . . 3
- 1.2 Differential equations . . . 3
- 1.3 The number e and the exponential function . . . 4
- 1.4 Vectors . . . 4
- 1.5 Matrices and matrix–vector multiplication . . . 4
- 1.6 The identity matrix . . . 5
- 1.7 The determinant . . . 5
- 1.8 The inverse of a matrix . . . 6
- 1.9 Complex numbers . . . 6

2 What a state is **6**

3 Systems with one state **7**
- 3.1 A bathtub . . . 7
- 3.2 Solving it: coffee and savings . . . 8

4 Systems with several states **9**
- 4.1 Two tubs . . . 9
- 4.2 Writing it as a matrix . . . 9
- 4.3 The general form . . . 9
- 4.4 What the equation means as a picture . . . 10

5 From a differential equation to state space **11**
5.1 Car suspension . . . 11
5.2 Any differential equation . . . 12

6 Eigenvalues and eigenvectors **12**
6.1 The easy case: states that do not affect each other . . . 12
6.2 The usual case: states that are mixed . . . 13
6.3 The definition . . . 13
6.4 Why this is the right idea . . . 14
6.5 How to find eigenvalues . . . 14
6.6 Step by step . . . 15
6.7 The solution from a starting point . . . 15
6.8 Four typical pictures . . . 16
6.9 Why eigenvalues are such a big deal . . . 16

7 Reading an eigenvalue **17**

8 Stability **18**
8.1 A shortcut for 2×2 matrices . . . 18

9 The suspension in numbers, and settling time **19**
9.1 The suspension in numbers . . . 19
9.2 Which mode matters most . . . 19

10 Transfer functions **20**
10.1 What a transfer function is . . . 20
10.2 From state space to a transfer function . . . 20
10.3 Poles are eigenvalues . . . 20
10.4 Zeros . . . 21
10.5 The two tubs . . . 21

11 Summary of Part 1 **22**

II The acrobot **22**

12 The acrobot and the three tasks **22**

13 Nonlinear systems and linear models **23**

14 Feedback, controllability and LQR **24**

15 The acrobot model **28**

16 Linear models and stability of the acrobot **29**

17 Acrobot controllers **31**

18 Summary of Part 2 **38**

19 Exercises **38**

A More detail on transfer functions **41**

Part I

State space and stability

1 Toolkit

1.1 Rates of change and the dot notation *slides 5*

Most of control engineering is about things that change over time: a temperature, a water level, the position of a car. We write time as t, measured in seconds.

If x is some quantity, the *rate of change* of x is how much x changes per second. If a tub's water level rises by 2 cm every second, its rate of change is 2 cm/s. If it falls by 2 cm every second, the rate of change is -2 cm/s.

Mathematicians write the rate of change as $\dfrac{dx}{dt}$, the *derivative* of x with respect to time. Engineers often use a shorter notation, a dot over the letter:

$$\dot{x} = \frac{dx}{dt}.$$

Read $\dot{x}$ as "x dot" or "the rate of change of x".

Two dots mean the rate of change of the rate of change:

$$\ddot{x} = \frac{d\dot{x}}{dt} = \frac{d^2x}{dt^2}.$$

The most familiar example is motion. If p is the position of a car, then $\dot{p}$ is its speed (how fast the position changes) and $\ddot{p}$ is its acceleration (how fast the speed changes).

You will also see $x(0)$, which means "the value of x at time $t = 0$", that is, at the start.

1.2 Differential equations

A *differential equation* is an equation that involves a quantity and its rates of change. For example,

$$\dot{x} = -2x$$

says: "the rate of change of x is always -2 times x". So when x is large and positive, it is falling quickly; when x is small, it is falling slowly.

Solving a differential equation means finding the function $x(t)$, that is, the value of x at every time, that makes the equation true. In physical systems the differential equation comes from a law of physics (conservation of mass, Newton's second law, and so on), and the solution tells us how the system behaves.

1.3 The number e and the exponential function

The number $e \approx 2.718$ shows up whenever something changes at a rate proportional to its own size. The function e^{at} (e raised to the power a times t) has a special property:

$$\frac{d}{dt}e^{at} = a\,e^{at}.$$

In words: the rate of change of e^{at} is a times itself. This is exactly why it solves equations like $\dot{x} = ax$.

What e^{at} looks like depends on the sign of a:

- If $a < 0$, then e^{at} starts at 1 and decays towards zero. For example e^{-t} is 1 at $t = 0$, 0.37 at $t = 1$, 0.14 at $t = 2$, 0.05 at $t = 3$.
- If $a > 0$, then e^{at} starts at 1 and grows faster and faster.
- If $a = 0$, then $e^0 = 1$ for all time.

Remember these three cases. Most of what we say about stability later comes back to them, with complex numbers added in Section 1.9.

1.4 Vectors *slides 11*

A *vector*, in this course, is just a list of numbers written in a column. We write vectors in bold:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}.$$

Here x_1 is the first entry and x_2 the second. A vector with n entries is called an n-vector. We use vectors to keep several related numbers together, such as the position and speed of a car.

Two vectors of the same size are added entry by entry, and a vector is multiplied by a number by multiplying every entry:

$$\begin{bmatrix} 1 \\ 2 \end{bmatrix} + \begin{bmatrix} 3 \\ 5 \end{bmatrix} = \begin{bmatrix} 4 \\ 7 \end{bmatrix}, \qquad 3\begin{bmatrix} 1 \\ 2 \end{bmatrix} = \begin{bmatrix} 3 \\ 6 \end{bmatrix}.$$

The rate of change of a vector is the vector of rates of change: $\dot{\mathbf{x}} = [\dot{x}_1,\ \dot{x}_2]^T$. (The little T means "transpose": it turns a row into a column, so we can write column vectors on one line.)

1.5 Matrices and matrix–vector multiplication *slides 11–12*

A *matrix* is a table of numbers. A matrix with n rows and m columns is called $n \times m$. We name the entry in row i and column j as A_{ij}:

$$A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}.$$

The one operation we need all the time is multiplying a matrix by a vector. The rule: for each row of the matrix, multiply its entries by the matching entries of the vector and add them up. That sum is one entry of the answer.

$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}\begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 1\cdot 5 + 2\cdot 6 \\ 3\cdot 5 + 4\cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}.$$

The number of columns of the matrix must equal the number of entries of the vector. An $n \times n$ matrix times an n-vector gives an n-vector.

In symbols, for a 2×2 matrix:

$$\begin{bmatrix} a & b \\ c & d \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} = \begin{bmatrix} ax_1 + bx_2 \\ cx_1 + dx_2 \end{bmatrix}.$$

Matrices of the same size are added entry by entry, and multiplied by a number entry by entry, just like vectors.

1.6 The identity matrix *slides 25*

The *identity matrix* I has 1s on the diagonal (top left to bottom right) and 0s elsewhere:

$$I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}.$$

It is the do-nothing matrix: multiplying any vector by I leaves it unchanged, $I\mathbf{v} = \mathbf{v}$. Check with the rule above: $1 \cdot v_1 + 0 \cdot v_2 = v_1$ and $0 \cdot v_1 + 1 \cdot v_2 = v_2$. Multiplying I by a number λ puts λ on the diagonal:

$$\lambda I = \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix}, \qquad \lambda I - A = \begin{bmatrix} \lambda - a & -b \\ -c & \lambda - d \end{bmatrix} \text{ for } A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}.$$

1.7 The determinant *slides 24*

The *determinant* of a 2×2 matrix is one number calculated from its four entries:

$$\det \begin{bmatrix} p & q \\ r & s \end{bmatrix} = ps - qr.$$

(Multiply down the main diagonal, subtract the product of the other diagonal.) For example $\det \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} = 4 - 6 = -2$.

The determinant has a picture. A 2×2 matrix sends each arrow in the plane to a new arrow, so it turns the square with corners $(0,0)$, $(1,0)$, $(0,1)$, $(1,1)$ into a slanted parallelogram. The square has area 1, and the parallelogram has area $|\det M|$. So the determinant says how much the matrix stretches areas. (A negative sign means the shape has also been flipped over.) If $\det M = 2$, every area doubles. If $\det M = 0$, the square has been squashed flat onto a line, with no area left.

What makes the determinant useful is this fact:

$$\det M = 0 \iff \text{there is a vector } \mathbf{v} \neq 0 \text{ with } M\mathbf{v} = 0.$$

Here is why, for 2×2. $M\mathbf{v} = 0$ means $pv_1 + qv_2 = 0$ and $rv_1 + sv_2 = 0$. If $ps - qr \neq 0$, the only solution is $v_1 = v_2 = 0$. If $ps - qr = 0$, the two rows are multiples of each other, so the two equations say the same thing and there is a whole line of non-zero solutions. In the picture: if M sends some non-zero arrow to zero, the whole plane gets flattened onto a line (or a point), so the area is 0. If the area is not 0, nothing has been flattened, so no non-zero arrow is sent to zero.

For bigger matrices the determinant is defined too (with a longer formula), and the same fact holds. In practice a computer calculates it.

1.8 The inverse of a matrix

For ordinary numbers, dividing by m is the same as multiplying by $1/m$, and $m \cdot (1/m) = 1$. Matrices have the same idea: the *inverse* M^{-1} of a square matrix M is the matrix with $M^{-1}M = I$. Multiplying by M^{-1} undoes multiplying by M. For a 2×2 matrix,

$$\begin{bmatrix} p & q \\ r & s \end{bmatrix}^{-1} = \frac{1}{ps - qr} \begin{bmatrix} s & -q \\ -r & p \end{bmatrix}$$

(swap the diagonal entries, change the sign of the other two, divide by the determinant). The inverse exists only when the determinant is not zero. For example,

$$\begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}^{-1} = \frac{1}{2-1} \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix} = \begin{bmatrix} 1 & -1 \\ -1 & 2 \end{bmatrix},$$

and you can check that multiplying the two gives I. We need inverses for transfer functions (Section 10) and again in Part 2.

1.9 Complex numbers *slides 31*

Some equations have no real solutions. For example $\lambda^2 = -1$: no real number squared gives -1. To handle this we introduce a new number j with

$$j^2 = -1, \qquad j = \sqrt{-1}.$$

(Mathematicians write i; engineers write j because i is used for electric current.)

A *complex number* has the form $\sigma + j\omega$, where σ and ω are ordinary real numbers. σ is the *real part* and ω is the *imaginary part*. We write $\mathrm{Re}(\sigma + j\omega) = \sigma$.

Complex numbers appear when we solve quadratic equations. The formula for the roots of $\lambda^2 + \beta\lambda + \gamma = 0$ is

$$\lambda = \frac{-\beta \pm \sqrt{\beta^2 - 4\gamma}}{2}.$$

If $\beta^2 - 4\gamma$ is negative, its square root is imaginary. For example, for $\lambda^2 + 2\lambda + 4 = 0$:

$$\lambda = \frac{-2 \pm \sqrt{4 - 16}}{2} = \frac{-2 \pm \sqrt{-12}}{2} = \frac{-2 \pm j\sqrt{12}}{2} = -1 \pm j\sqrt{3}.$$

Notice the roots come as a pair, $-1 + j\sqrt{3}$ and $-1 - j\sqrt{3}$. Complex roots of an equation with real coefficients always come in such pairs, called *conjugate pairs*.

The other fact we need is *Euler's formula*, which links exponentials to sines and cosines:

$$e^{j\omega t} = \cos\omega t + j \sin\omega t.$$

So an exponential with an imaginary power is an oscillation. When a conjugate pair appears in a physical solution, the two terms combine and the j parts cancel, leaving an ordinary real oscillation such as $\cos\omega t$.

2 What a state is *slides 6–7*

Two cars are both at the 3 km marker on a road. One is parked and the other is doing 60 km/h. A minute later the first is still at km 3 and the second is at km 4. So knowing the position alone does not tell you where a car will be. If I tell you a car is at km 3 *and* doing 60 km/h, you can

predict that it will be at km 4 (assuming you also know what the driver does with the pedals). Knowing more, such as the colour of the car, would not help.

The *state* of a system is the smallest set of numbers that, together with the inputs from now on, lets you predict everything the system will do. For the car, the state is position and speed. We collect the state into a vector $\mathbf{x}$:

$$\mathbf{x} = \begin{bmatrix} \text{position} \\ \text{speed} \end{bmatrix}.$$

A useful picture is a video-game save file: it stores exactly what is needed to carry on the game from where it stopped.

How many numbers do we need? A cup of hot coffee needs one, its temperature. A bathtub needs one, its water level. A playground swing needs two, its angle and its speed. Each of these numbers describes energy stored somewhere: heat in the coffee, the height of water in the tub, the height and motion of the swing. Stored energy cannot change instantly (you cannot heat coffee or fill a tub in zero time), so the system has to "remember" it. That gives a rule of thumb:

$$\text{number of states} = \text{number of separate places energy is stored.}$$

This is also the *order* of the system: a first-order system has one state, a second-order system has two.

Common energy stores and the state that goes with each: a hot object (temperature), a tank (level), something lifted against gravity (height or angle), a spring (how much it is stretched), a moving mass (speed), a capacitor (voltage), an inductor (current).

3 Systems with one state

slides 8–9

3.1 A bathtub

Picture a bathtub with the tap running and the plug out. Water comes in from the tap and goes out through the drain. The fuller the tub, the higher the water pressure at the drain, so the faster it drains.

Let us name everything:

- h: the water level in metres. This is the state.
- $\dot{h}$: the rate at which the level changes, in m/s.
- q_{in}: the flow from the tap in m^3/s. This is the input, the thing we control.
- A_t: the floor area of the tub in m^2 (assume straight sides).
- R: the drain resistance. We model the outflow as h/R: double the level, double the outflow.

The straight-line drain model is a simplification. Water pouring freely out of a hole follows Torricelli's law, where the outflow grows like $\sqrt{h}$. The model h/R is accurate for slow flow through a narrow pipe, and otherwise it is a good approximation for small changes around one level. Strictly, the straight-line fit is for changes: near a normal level, a small rise in level gives a proportional rise in outflow. We use it because it keeps the equation linear.

The volume of water is $A_t h$, so the rate of change of volume is $A_t \dot{h}$. Water is not created or destroyed, and its density stays the same, so conservation of mass becomes a balance of volumes: the rate of change of volume equals the flow in minus the flow out:

$$A_t \, \dot{h} = q_{\text{in}} - \frac{h}{R}.$$

Divide both sides by A_t:

$$\dot{h} = -\frac{1}{A_t R}\, h + \frac{1}{A_t}\, q_{\text{in}}.$$

To save writing, give the two constant combinations names: $a = -1/(A_t R)$ and $b = 1/A_t$. Writing x for the state and u for the input,

$$\dot{x} = a\, x + b\, u.$$

Read this as: "the rate of change of the state is a times the state plus b times the input." The term ax is what the system does by itself (the tub drains, faster when fuller; a is minus the drain's number $1/(A_t R)$, which is why it is negative). The term bu is the effect of the input (the tap).

The combination $T = A_t R$ has units of seconds and is called the *time constant*; then $a = -1/T$. If the tap is held at a steady flow, the level stops changing when $\dot{h} = 0$, which gives $h = Rq_{\text{in}}$: that is where it settles.

3.2 Solving it: coffee and savings

To see what a does, turn the input off ($u = 0$), leaving $\dot{x} = ax$.

Two everyday examples have this form. A cup of coffee cools faster when it is much hotter than the room, and more slowly as it approaches room temperature; if x is the temperature above room temperature, $\dot{x} = ax$ with $a < 0$ (Newton's law of cooling). Money in a savings account earns more interest the more there is; if x is the balance and interest is added continuously, $\dot{x} = ax$ with $a > 0$. In both cases the rate of change is proportional to the amount. That means that every second the amount changes by the same fraction of itself: the coffee's extra heat might drop by 1% each second, whether it is 50 or 5 degrees above the room. Changing by the same fraction in every equal step of time is exactly what an exponential e^{at} does.

Solving $\dot{x} = ax$. Write it as $\dfrac{dx}{dt} = ax$ and move the x's to one side and the t's to the other:

$$\frac{dx}{x} = a\, dt.$$

Integrate both sides (the integral of $1/x$ is $\ln x$, the natural logarithm):

$$\ln x = at + \text{constant}.$$

Take e to the power of both sides: $x = e^{\text{constant}}\, e^{at}$. At $t = 0$, $e^{at} = 1$, so e^{constant} must be $x(0)$. Therefore

$$x(t) = e^{at}\, x(0).$$

You can check this directly using the property from Section 1.3: $\dot{x} = ae^{at}x(0) = ax$.

What it means.

- $a < 0$ (coffee): x decays to zero. The system returns to rest by itself. We call this *stable*.
- $a > 0$ (savings): x grows without limit. We call this *unstable*.
- $a = 0$: x stays wherever it starts. We call this *marginally stable*.

For one state, a single number a tells you everything. If $a = -1/T$, then after one time constant T the state has fallen to $e^{-1} \approx 37\%$ of its starting value, and after $4T$ to about 2%.

The rest of the lecture answers one question: *when there are several states, what plays the role of a?* The answer is the eigenvalues.

4 Systems with several states

slides 10–18

4.1 Two tubs

Put two tubs one above the other. The tap fills tub 1, tub 1 drains into tub 2, and tub 2 drains away. Let h_1 and h_2 be the two levels and q the tap flow. To keep the numbers simple, take all areas and resistances equal to 1, except that tub 2 drains twice as fast. Applying the bathtub equation to each tub:

$$\dot{h}_1 = -h_1 + q \qquad \text{(in from tap, out to tub 2)}$$
$$\dot{h}_2 = h_1 - 2h_2 \qquad \text{(in from tub 1, out through its own drain)}$$

Each line is the single bathtub again. The new feature is the h_1 in the second line: what happens in tub 1 affects tub 2. The states are *coupled*.

4.2 Writing it as a matrix

Look at the right-hand sides. Each one is a sum of "a number times h_1", "a number times h_2" and "a number times q". Fill in the missing terms with zeros:

$$\dot{h}_1 = (-1)\,h_1 + (0)\,h_2 + (1)\,q$$
$$\dot{h}_2 = (1)\,h_1 + (-2)\,h_2 + (0)\,q$$

Compare with the matrix–vector rule in Section 1.5. The numbers multiplying h_1 and h_2 form a matrix and the numbers multiplying q form a vector:

$$\begin{bmatrix}\dot{h}_1\\ \dot{h}_2\end{bmatrix} = \begin{bmatrix}-1 & 0\\ 1 & -2\end{bmatrix}\begin{bmatrix}h_1\\ h_2\end{bmatrix} + \begin{bmatrix}1\\ 0\end{bmatrix} q.$$

If you multiply this out row by row you get the two original equations back. The matrix form contains nothing new; it is just a compact way of writing several equations at once.

Reading the entries. Call the matrix A. The entry A_{ij} (row i, column j) says how strongly state j affects the rate of change of state i.

- $A_{11} = -1$: tub 1 drains by itself.
- $A_{22} = -2$: tub 2 drains by itself, twice as fast.
- $A_{21} = 1$: tub 1 fills tub 2.
- $A_{12} = 0$: tub 2 does not affect tub 1 (water does not flow uphill).

The diagonal entries describe each state on its own; the off-diagonal entries describe how the states affect each other. The input vector $B = [1,\ 0]^T$ says the tap only feeds tub 1.

4.3 The general form

Any system of this kind can be written as

$$\dot{\mathbf{x}} = A\mathbf{x} + Bu, \qquad y = C\mathbf{x} + Du.$$

Here is what every symbol means.

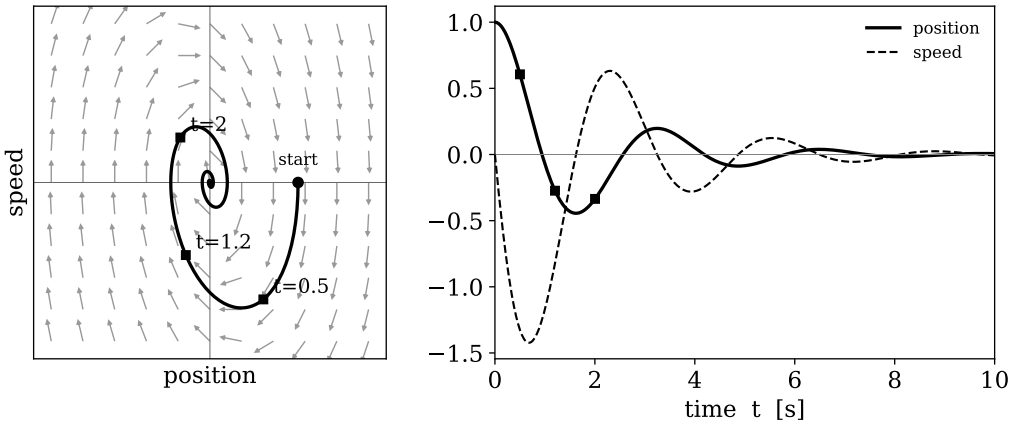
- $\mathbf{x}$: the state vector, with n entries (one per energy store).
- $\dot{\mathbf{x}}$: the vector of rates of change of the states.
- u: the input, the thing we can control (tap flow, motor voltage, force).

- y: the output, the thing we measure (it might be one of the states, or a combination).
- A: an $n \times n$ matrix describing what the system does by itself. Row i tells you how the rate of change of state i depends on all the states.
- B: an $n \times 1$ vector describing how the input pushes on each state.
- C: a $1 \times n$ row describing what combination of states we measure.
- D: a single number for any direct effect of the input on the output; usually zero.

This is called the *state-space* form. The first equation (the *state equation*) describes the dynamics. The second (the *output equation*) just says what we measure.

An important point: whether the system settles down by itself depends only on A. In the tub, whether the level settles depends on the drain, not on how you use the tap. So for stability we only need to study A.

4.4 What the equation means as a picture *slides 15–18*



How to read these pictures. They show up several times, so it is worth being clear about what is in them. With two states, think of the state as a point on a flat plane: the first state across, the second up. For the car suspension that is position across and speed up, so every spot on the plane is one possible situation the car could be in (for example "0.5 m up and moving down at 1 m/s").

- **The small grey arrows.** At each spot $\mathbf{x}$, the equation gives $\dot{\mathbf{x}} = A\mathbf{x}$: which way the state is about to move from there. Each little arrow is drawn pointing in that direction. (They are all drawn the same length, so they only show direction.) Together they are a map: wherever the system is, the arrow there tells you where it goes next.

- **The thick line.** This is one actual motion. It starts at the dot and follows the arrows. It is what you would see if you started the car at that position and speed and let go.

- **The time plot** (right-hand side in the first figure, bottom row in the four-case figure). This is the same motion drawn the usual way, against time. The small squares mark the same moments in both plots.

- **Dashed lines**, where they appear, are the special directions (eigenvectors) discussed in Section 6.

In the first figure, the suspension starts raised above its rest height (position 1, speed 0). It moves down, speeds up, overshoots, comes back, and each loop is smaller: the spiral in the left

plot and the dying wiggle in the right plot are the same thing.

This also gives the simplest way to simulate a system on a computer. Over a small time step Δt (say 0.01 s), the state moves a little way along its arrow:

$$\mathbf{x}(t + \Delta t) \approx \mathbf{x}(t) + \Delta t \left(A\mathbf{x}(t) + Bu(t)\right).$$

For example, take the suspension matrix $A = \begin{bmatrix} 0 & 1 \\ -4 & -1 \end{bmatrix}$, no input, start at $\mathbf{x} = [1,\ 0]^T$ and use $\Delta t = 0.1$. Then $A\mathbf{x} = [0,\ -4]^T$, so the next state is $[1,\ 0]^T + 0.1\,[0,\ -4]^T = [1,\ -0.4]^T$. From there $A\mathbf{x} = [-0.4,\ -3.6]^T$, so the next state is $[0.96,\ -0.76]^T$. The body has started moving down and is picking up speed. Repeating this thousands of times traces out the spiral. (This is Euler's method; real simulators use more accurate versions of the same idea.)

5 From a differential equation to state space

slides 14, 19

5.1 Car suspension

A car body sits on a spring and a shock absorber (damper). Name everything:

- p: how far the body is above its resting height, in metres. At rest the spring is already squashed a little and holds up the car's weight, so measuring p from there means gravity is already balanced and drops out of the equation.
- m: mass of the body, in kg.
- k: spring stiffness, in N/m. If the body is p metres above rest, the spring pulls it back with force kp. A stiffer spring has a larger k.
- c: damping coefficient, in N s/m. The shock absorber resists motion with a force proportional to speed, $c\dot{p}$. A stronger damper has a larger c.
- u: any extra force pushing on the body, in N (the input).

Newton's second law says mass times acceleration equals total force. The spring force is $-kp$ (it pulls back towards rest, hence the minus sign), the damper force is $-c\dot{p}$ (it opposes motion), and the input is u:

$$m\ddot{p} = -kp - c\dot{p} + u, \qquad \text{or} \qquad m\ddot{p} + c\dot{p} + kp = u.$$

This is one second-order differential equation (it contains $\ddot{p}$). To get state-space form, pick the states from the energy stores: the spring stores energy through its stretch p, and the mass stores energy through its speed $\dot{p}$. So

$$x_1 = p, \qquad x_2 = \dot{p}.$$

Now write the rate of change of each state using only x_1, x_2 and u:

- $\dot{x}_1 = \dot{p} = x_2$. (The rate of change of position is the speed. This one is free.)
- $\dot{x}_2 = \ddot{p}$. From Newton, $\ddot{p} = -\frac{k}{m}p - \frac{c}{m}\dot{p} + \frac{1}{m}u = -\frac{k}{m}x_1 - \frac{c}{m}x_2 + \frac{1}{m}u$.

Writing $v = \dot{p}$ for the speed and filling in the missing terms, the two lines are

$$\dot{p} = 0 \cdot p + 1 \cdot v, \qquad \dot{v} = -\tfrac{k}{m}\,p - \tfrac{c}{m}\,v + \tfrac{1}{m}\,u,$$

and each row of A is one of these lines. Putting the numbers in matrix form:

$$\begin{bmatrix} \dot{x}_1 \\ \dot{x}_2 \end{bmatrix} = \underbrace{\begin{bmatrix} 0 & 1 \\ -\frac{k}{m} & -\frac{c}{m} \end{bmatrix}}_{A} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + \underbrace{\begin{bmatrix} 0 \\ \frac{1}{m} \end{bmatrix}}_{B} u, \qquad y = \underbrace{\begin{bmatrix} 1 & 0 \end{bmatrix}}_{C}\mathbf{x} \ \text{(we measure position)}.$$

Read the matrix: the top row says position changes at the speed; the bottom row is Newton's law. $-k/m$ is how strongly the position changes the speed, through the spring; $-c/m$ is how strongly the damper slows the speed down. $1/m$ in B says a force changes the speed more if the mass is small.

Every mechanical system looks like this: states are positions and speeds, the top rows say positions change at the speeds, and the bottom rows are Newton's law.

5.2 Any differential equation

The same trick works for any differential equation. Take a general second-order equation

$$\ddot{y} + a_1\dot{y} + a_0 y = b\,u,$$

where y is the quantity we care about, a_0 and a_1 are constant numbers that come from the physics, and b says how strongly the input enters. (For the suspension, divide by m: $a_1 = c/m$, $a_0 = k/m$, $b = 1/m$.)

Choose the variable and its derivative as states, $x_1 = y$ and $x_2 = \dot{y}$. Then $\dot{x}_1 = x_2$, and solving the equation for the highest derivative gives $\dot{x}_2 = \ddot{y} = -a_0 x_1 - a_1 x_2 + bu$:

$$A = \begin{bmatrix} 0 & 1 \\ -a_0 & -a_1 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ b \end{bmatrix}, \qquad C = \begin{bmatrix} 1 & 0 \end{bmatrix}.$$

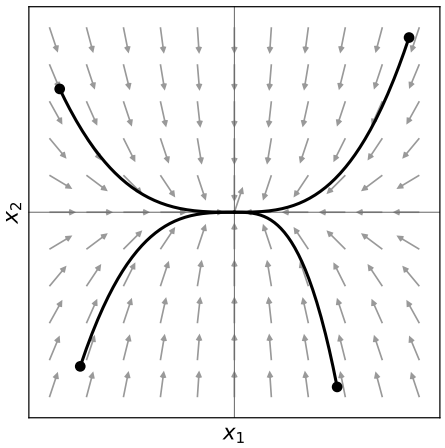
For an nth-order equation (highest derivative $y^{(n)}$, the nth derivative), use the n states y, $\dot{y}$, $\ddot{y}$, …, $y^{(n-1)}$. The top rows of A each say "my rate of change is the next state", and the bottom row is the equation solved for $y^{(n)}$.

One rule always applies when you write state equations: the right-hand side may contain only states and inputs, never a derivative. If a derivative appears, a state is missing.

6 Eigenvalues and eigenvectors *slides 20–30*

We now return to the question from Section 3: with several states, what plays the role of a?

6.1 The easy case: states that do not affect each other

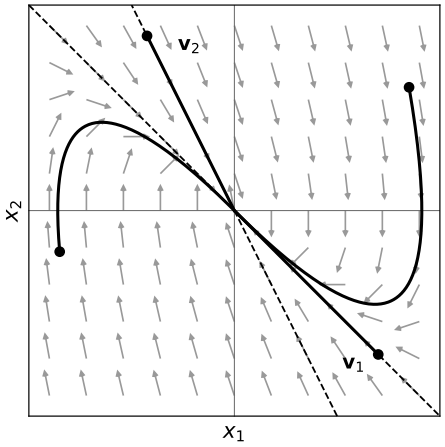


Suppose

$$A = \begin{bmatrix} -1 & 0 \\ 0 & -3 \end{bmatrix}.$$

Multiplying out, $\dot{x}_1 = -x_1$ and $\dot{x}_2 = -3x_2$. Neither equation mentions the other state. Each state is its own cup of coffee: from Section 3, $x_1(t) = e^{-t}x_1(0)$ and $x_2(t) = e^{-3t}x_2(0)$. The decay rates are simply the numbers on the diagonal of A. In the picture every trajectory slides into the origin, quicker in the x_2 direction (rate 3) than in the x_1 direction (rate 1).

6.2 The usual case: states that are mixed



Usually A has non-zero off-diagonal entries, for example

$$A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}.$$

Now $\dot{x}_1 = x_2$ and $\dot{x}_2 = -2x_1 - 3x_2$, and each state's rate of change depends on the other. The states are mixed, so we cannot treat them as separate cups.

But look at the arrows in the picture. Along two particular lines through the origin (the dashed lines), the arrows point exactly along the line. If the system starts on one of these lines, it stays on that line and simply shrinks towards the origin at a single rate, just like one cup of coffee. Starting anywhere else, the motion is a mixture of the two.

These special directions are called **eigenvectors** of A, and the rate of shrinking or growing along each one is its **eigenvalue**. ("Eigen" is German for "own": the matrix's own directions and own rates.) If we used the eigenvectors as our axes instead of x_1 and x_2, the system would split into separate cups again. That is the whole idea: the eigenvectors undo the mixing.

A car gives an everyday example (slide 23). After a bump, a car body can bounce (front and back go up and down together) or pitch (the nose dips while the tail rises). For a car with its weight evenly balanced between front and back, pushing it down evenly and letting go gives pure bounce, which keeps its shape and just dies away. Pushing the nose down and lifting the tail by the same amount, then letting go, gives pure pitch: the car rocks about its middle. Bounce and pitch are then the car's special directions, each with its own frequency and decay rate, and any other bump produces a mix of the two. (The car is only a picture of the idea. Its matrix is not the 2×2 example above.)

6.3 The definition

In symbols: a vector $\mathbf{v}$ is an eigenvector of A, with eigenvalue λ (the Greek letter lambda), if

$$A\mathbf{v} = \lambda\mathbf{v}, \qquad \mathbf{v} \neq 0.$$

In words: multiplying $\mathbf{v}$ by A gives back the same vector, only stretched or shrunk by the number λ. That is exactly the statement "the arrow at $\mathbf{v}$ points along $\mathbf{v}$".

We can check this for the example above with $\mathbf{v} = [1,\ -1]^T$:

$$\begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} \begin{bmatrix} 1 \\ -1 \end{bmatrix} = \begin{bmatrix} 0-1 \\ -2+3 \end{bmatrix} = \begin{bmatrix} -1 \\ 1 \end{bmatrix} = -1 \cdot \begin{bmatrix} 1 \\ -1 \end{bmatrix}.$$

So $[1,\ -1]^T$ is an eigenvector with eigenvalue -1.

6.4 Why this is the right idea *slides 25*

Suppose the system starts on an eigenvector, $\mathbf{x}(0) = \mathbf{v}$. Since it stays on that line and changes at one rate, guess the solution

$$\mathbf{x}(t) = e^{\lambda t}\mathbf{v}.$$

Check it in $\dot{\mathbf{x}} = A\mathbf{x}$. The left side: $\dot{\mathbf{x}} = \lambda e^{\lambda t}\mathbf{v}$ (only the $e^{\lambda t}$ depends on time). The right side: $A\mathbf{x} = e^{\lambda t}A\mathbf{v}$. These are equal exactly when $A\mathbf{v} = \lambda\mathbf{v}$. So along an eigenvector, the state behaves like $e^{\lambda t}$, exactly like the one-state case with a replaced by λ.

A general starting point can be written as a mixture of the eigenvectors, $\mathbf{x}(0) = c_1\mathbf{v}_1 + c_2\mathbf{v}_2 + \cdots + c_n\mathbf{v}_n$, where the numbers c_i say how much of each direction is in the starting point. (This is always possible when the eigenvalues are all different, which is the usual case. Repeated eigenvalues can also add terms like $t\,e^{\lambda t}$.) Each part then evolves on its own:

$$\mathbf{x}(t) = c_1 e^{\lambda_1 t}\mathbf{v}_1 + c_2 e^{\lambda_2 t}\mathbf{v}_2 + \cdots + c_n e^{\lambda_n t}\mathbf{v}_n.$$

(This works because the equation is linear: if two motions each satisfy $\dot{\mathbf{x}} = A\mathbf{x}$, so does their sum.) Each term is called a *mode* of the system.

This is the key result. Every motion of the system is a sum of terms $e^{\lambda_i t}$. If every one of them decays, the system settles. If even one grows, it does not.

6.5 How to find eigenvalues

Rewrite $A\mathbf{v} = \lambda\mathbf{v}$ as $A\mathbf{v} = \lambda I\mathbf{v}$ (since $I\mathbf{v} = \mathbf{v}$), then move everything to one side:

$$(\lambda I - A)\mathbf{v} = 0.$$

We want a solution with $\mathbf{v} \neq 0$. By the determinant fact in Section 1.7, this happens exactly when

$$\det(\lambda I - A) = 0.$$

Here is the picture behind this. If $\mathbf{v}$ is a special direction, then $\lambda I - A$ sends the non-zero arrow $\mathbf{v}$ to zero: it crushes it. A matrix that crushes a non-zero arrow flattens the whole plane onto a line, so its area (its determinant) is 0. Now slide λ along and watch the area. For $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$, the matrix $\lambda I - A$ has columns $[\lambda,\ 2]^T$ and $[-1,\ \lambda+3]^T$, and its area is $\lambda^2 + 3\lambda + 2$:

- at $\lambda = 0$ the area is 2;
- at $\lambda = -0.5$ it is 0.75;
- at $\lambda = -1$ it is 0. Both columns are now $[-1,\ 2]^T$, so the square is squashed onto a line.

So $\lambda = -1$ is special, and the arrow that gets crushed, here $[1,\ -1]^T$, is its special direction. The other way round, if $\det(\lambda I - A) \neq 0$, nothing is flattened, so no non-zero arrow is crushed and that λ is not an eigenvalue.

The equation $\det(\lambda I - A) = 0$ is called the *characteristic equation*. For a 2×2 matrix it is a quadratic in λ, so there are two eigenvalues. For an $n \times n$ matrix there are n, counting repeats.

6.6 Step by step *slides 26*

Take $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$.

Step 1. Form $\lambda I - A$:

$$\lambda I - A = \begin{bmatrix} \lambda & 0 \\ 0 & \lambda \end{bmatrix} - \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix} = \begin{bmatrix} \lambda & -1 \\ 2 & \lambda + 3 \end{bmatrix}.$$

Step 2. Take the determinant ($ps - qr$):

$$\det(\lambda I - A) = \lambda(\lambda + 3) - (-1)(2) = \lambda^2 + 3\lambda + 2.$$

Step 3. Set it to zero and solve. It factorises as $(\lambda + 1)(\lambda + 2) = 0$, so

$$\lambda_1 = -1, \qquad \lambda_2 = -2.$$

Step 4. For each eigenvalue, find the eigenvector by solving $(A - \lambda I)\mathbf{v} = 0$. (This is the same as $(\lambda I - A)\mathbf{v} = 0$ with the sign flipped, which does not change the solutions.)

For $\lambda_1 = -1$: $A + I = \begin{bmatrix} 1 & 1 \\ -2 & -2 \end{bmatrix}$. The first row says $v_1 + v_2 = 0$; the second row says the same thing (it is -2 times the first, as expected since the determinant is zero). Any vector with $v_2 = -v_1$ works; pick $\mathbf{v}_1 = [1,\ -1]^T$.

For $\lambda_2 = -2$: $A + 2I = \begin{bmatrix} 2 & 1 \\ -2 & -1 \end{bmatrix}$. The first row says $2v_1 + v_2 = 0$, so $v_2 = -2v_1$; pick $\mathbf{v}_2 = [1,\ -2]^T$.

Eigenvectors are only defined up to length: $[2,\ -2]^T$ is just as good as $[1,\ -1]^T$. What matters is the direction. These two directions are the dashed lines in the mixed-states picture.

6.7 The solution from a starting point *slides 27*

Suppose the system starts at $\mathbf{x}(0) = [1,\ 0]^T$. Write this as a mixture of the eigenvectors $\mathbf{v}_1 = [1,\ -1]^T$ and $\mathbf{v}_2 = [1,\ -2]^T$:

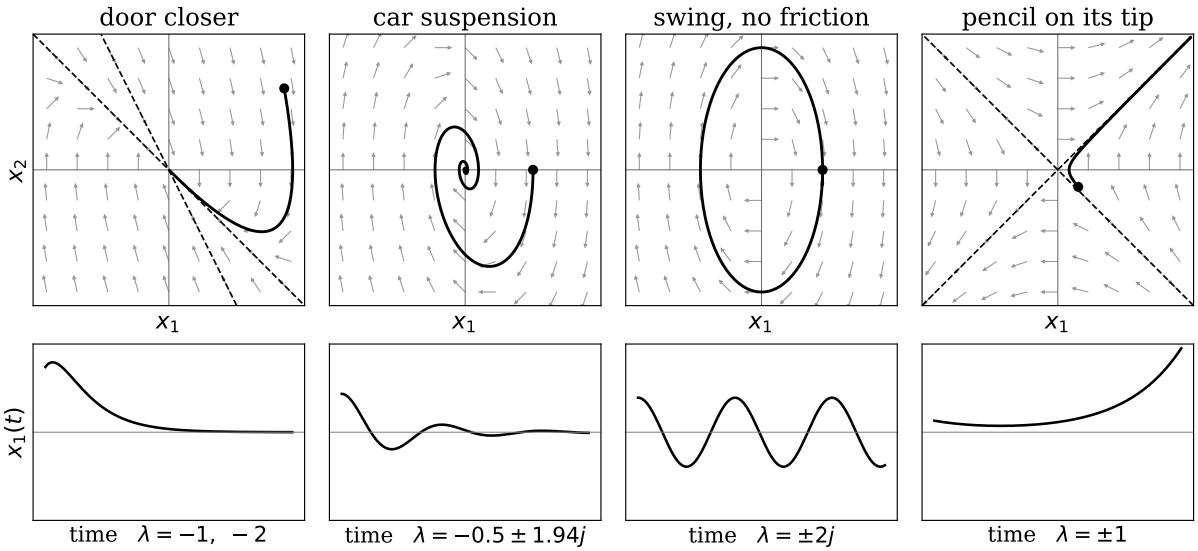
$$c_1 \begin{bmatrix} 1 \\ -1 \end{bmatrix} + c_2 \begin{bmatrix} 1 \\ -2 \end{bmatrix} = \begin{bmatrix} 1 \\ 0 \end{bmatrix} \quad \Longrightarrow \quad c_1 + c_2 = 1, \ \ -c_1 - 2c_2 = 0.$$

From the second equation $c_1 = -2c_2$; substituting in the first gives $-c_2 = 1$, so $c_2 = -1$ and $c_1 = 2$. Then

$$\mathbf{x}(t) = 2e^{-t} \begin{bmatrix} 1 \\ -1 \end{bmatrix} - e^{-2t} \begin{bmatrix} 1 \\ -2 \end{bmatrix}, \qquad x_1(t) = 2e^{-t} - e^{-2t}, \quad x_2(t) = -2e^{-t} + 2e^{-2t}.$$

So $\mathbf{x}(0) = 2\mathbf{v}_1 - \mathbf{v}_2$, and each part fades at its own rate. Check: at $t = 0$, $x_1 = 2 - 1 = 1$ and $x_2 = -2 + 2 = 0$. Correct. Both terms decay, so the system settles. The e^{-t} term decays more slowly than e^{-2t}, so it is the one that decides how long the system takes to settle.

6.8 Four typical pictures *slides 28–29*



For two states there are only a few kinds of behaviour. In the figure, the top row shows the arrows and one motion for each case, and the bottom row shows the first state of that same motion against time.

- **Two negative real eigenvalues** (like a door closer, $\lambda = -1, -2$): trajectories slide into the origin without circling.
- **A complex pair with negative real part** (like a car suspension, $\lambda = -0.5 \pm j1.94$): there are no real special directions, and trajectories spiral in. The spiralling is oscillation.
- **A purely imaginary pair** (like a swing with no friction, $\lambda = \pm j2$): trajectories go round in closed loops forever.
- **One positive and one negative real eigenvalue** (like a pencil balanced on its tip, $\lambda = \pm 1$): along one eigenvector the state shrinks, along the other it grows. Almost every starting point ends up running away along the growing direction.

6.9 Why eigenvalues are such a big deal *slides 30*

It is worth stopping here, because this is the part people (including lecturers) most often find slippery.

A matrix usually mixes things up. Take any vector $\mathbf{x}$ and multiply it by A. In general the result points in a different direction: A stretches it *and* turns it. In system terms, that turning is the coupling: the rate of change of x_1 depends on x_2 and vice versa, so you cannot work out what one state does without knowing the others. That is what makes a system with several states hard.

Along an eigenvector, it doesn't. An eigenvector is a direction that A does not turn, only stretches by the number λ. If the state lies along that direction, the arrow $A\mathbf{x}$ points along the same direction, so the state stays on that line and just shrinks or grows. All the coupling has disappeared, and what is left is the one-state equation $\dot{x} = \lambda x$ from Section 3, whose answer is $e^{\lambda t}$.

And everything else is a mix of those. This is the key step. With two states, write the

state in terms of the eigenvectors instead of in terms of x_1 and x_2:

$$\mathbf{x}(t) = c_1(t)\,\mathbf{v}_1 + c_2(t)\,\mathbf{v}_2.$$

Substitute this into $\dot{\mathbf{x}} = A\mathbf{x}$. The left side is $\dot{c}_1\mathbf{v}_1 + \dot{c}_2\mathbf{v}_2$. The right side is $c_1A\mathbf{v}_1 + c_2A\mathbf{v}_2 = c_1\lambda_1\mathbf{v}_1 + c_2\lambda_2\mathbf{v}_2$. Matching the amounts of $\mathbf{v}_1$ and $\mathbf{v}_2$ on both sides:

$$\dot{c}_1 = \lambda_1 c_1, \qquad \dot{c}_2 = \lambda_2 c_2.$$

So in these new coordinates the system is two separate cups of coffee, each with its own rate. The mixing depends on which axes we use to describe the state. Along the eigenvector axes it disappears. Choosing the eigenvectors as axes untangles it completely. For the example matrix in this section, the tangled equations $\dot{x}_1 = x_2$, $\dot{x}_2 = -2x_1 - 3x_2$ become simply $\dot{c}_1 = -c_1$ and $\dot{c}_2 = -2c_2$.

What that buys us. Three things follow.

- The eigenvalues are the complete list of ways the system can move. Any motion, from any starting point, is built from $e^{\lambda_1 t}, e^{\lambda_2 t}, \ldots$ and nothing else. If you know the eigenvalues, you know every possible behaviour.
- So stability is decided by the eigenvalues alone. If every $e^{\lambda_i t}$ dies away, every possible motion dies away.
- The eigenvalues belong to the system, not to how you wrote it down. If you measure position in centimetres instead of metres, or list the states in a different order, A changes but its eigenvalues do not. (Mathematically, changing variables replaces A by $P^{-1}AP$ for some invertible matrix P, and $\det(\lambda I - P^{-1}AP) = \det(\lambda I - A)$.) That is why it makes sense to talk about "the" eigenvalues of a car suspension.

A useful picture is a musical chord. Heard all at once it is one complicated sound, but it is really a few pure notes played together. The eigenvectors are the individual notes, the eigenvalues say how each note fades (or grows) and what pitch it has, and the system's motion is the chord.

7 Reading an eigenvalue

slides 32–33

Eigenvalues can be complex numbers (Section 1.9). This happens when solving $\det(\lambda I - A) = 0$ needs the square root of a negative number. Take the suspension with no shock absorber, $m = 1$, $k = 4$, $c = 0$:

$$A = \begin{bmatrix} 0 & 1 \\ -4 & 0 \end{bmatrix}, \qquad \det(\lambda I - A) = \lambda^2 + 4 = 0, \qquad \lambda = \pm\sqrt{-4} = \pm 2j.$$

Why does a j mean oscillation? Draw a complex number $\sigma + j\omega$ as an arrow on a plane, σ across and ω up. Multiplying by j turns the arrow a quarter turn: $j(\sigma + j\omega) = -\omega + j\sigma$. So in $\dot{x} = 2j\,x$ the rate of change is always at right angles to x itself. Moving always sideways to where you are, you never get nearer to the middle or further away. You go round in a circle, and going round a circle is oscillating.

In general, write an eigenvalue as $\lambda = \sigma + j\omega$, where σ (sigma) is the real part and ω (omega) is the imaginary part. Then, using $e^{p+q} = e^p e^q$ and Euler's formula,

$$e^{\lambda t} = e^{(\sigma + j\omega)t} = e^{\sigma t}\,e^{j\omega t} = e^{\sigma t}(\cos\omega t + j\sin\omega t).$$

So each mode is an oscillation $\cos\omega t$ inside an envelope $e^{\sigma t}$:

- The real part σ (units 1/s) decides growth or decay. $\sigma < 0$: the mode fades. $\sigma > 0$: it grows. Think of it as a volume knob.
- The imaginary part ω (units rad/s) is how fast the mode oscillates. $\omega = 0$: no oscillation. Think of it as the pitch of a note.

Because complex eigenvalues of a real matrix come in conjugate pairs $\sigma \pm j\omega$, the two j parts cancel when combined, and the actual motion is real: something like $e^{\sigma t}\cos\omega t$.

Three everyday examples fit these cases. A door closer glides shut without swinging: real negative eigenvalues ($\sigma < 0$, $\omega = 0$). A car with worn shock absorbers keeps bouncing, a little less each time: a complex pair with small negative σ. A microphone held near its speaker produces a tone that gets louder and louder: positive σ, non-zero ω.

Plotting eigenvalues as points on a plane, with σ across and ω up, gives a map (slide 33). Points further left fade faster. Points further from the horizontal axis oscillate faster. Anything in the right half grows.

8 Stability

slides 34–35

A marble at the bottom of a bowl, a marble on a flat table, and a marble balanced on top of an upturned bowl can all sit perfectly still. The difference is what happens after a small nudge. In the bowl it rolls back. On the table it moves to a new spot and stays there. On the upturned bowl it rolls off. *Stability* is about the response to a small disturbance, not about being at rest.

Since every motion is a sum of terms $e^{\lambda_i t}$:

- **Stable**: every eigenvalue has negative real part. Every mode fades; the system returns to rest. (Marble in a bowl with a bit of friction. In a frictionless bowl it would roll back and forth forever, which is marginal.)
- **Marginally stable**: some eigenvalues have zero real part (and are not repeated), and none have positive real part. Those modes neither fade nor grow. (Marble on a table, or a frictionless swing.) The "not repeated" matters: a repeated eigenvalue on the imaginary axis can let the state drift off. For example $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ has $\lambda = 0$ twice, and $\dot{x}_1 = x_2$ with x_2 constant gives $x_1 = x_1(0) + x_2(0)\,t$, which keeps growing.
- **Unstable**: at least one eigenvalue has positive real part. That mode grows without limit. (Marble on the upturned bowl.)

One growing mode is enough to make the whole system unstable, however well behaved the others are. And marginally stable is not the same as stable: a frictionless swing never stops.

8.1 A shortcut for 2×2 matrices

For $A = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$ (here a, b, c, d are just the four entries, not the bathtub's a and b):

$$\det(\lambda I - A) = \det \begin{bmatrix} \lambda - a & -b \\ -c & \lambda - d \end{bmatrix} = (\lambda - a)(\lambda - d) - bc = \lambda^2 - (a + d)\lambda + (ad - bc).$$

The number $a + d$, the sum of the diagonal entries, is called the *trace* of A, written $\operatorname{tr} A$. The number $ad - bc$ is $\det A$. So

$$\det(\lambda I - A) = \lambda^2 - (\operatorname{tr} A)\lambda + \det A.$$

If the two eigenvalues are λ_1 and λ_2, this quadratic equals $(\lambda-\lambda_1)(\lambda-\lambda_2) = \lambda^2-(\lambda_1+\lambda_2)\lambda+\lambda_1\lambda_2$. Comparing,

$$\lambda_1 + \lambda_2 = \operatorname{tr} A, \qquad \lambda_1\lambda_2 = \det A.$$

This is like guessing two numbers from their sum and their product. If the product is positive, the two have the same sign; if the sum is also negative, both are negative, that is, both on the left.

In more detail: if both eigenvalues are real and negative, their sum is negative and their product positive. If they are a complex pair $\sigma \pm j\omega$, their sum is 2σ and their product is $\sigma^2+\omega^2$, which is always positive, so the sum is negative exactly when $\sigma < 0$. Working through the other cases shows the reverse is also true. So

$$\text{stable} \iff \operatorname{tr} A < 0 \quad \text{and} \quad \det A > 0.$$

This needs no solving at all. Two extra facts: the eigenvalues are complex (oscillation) when $(\operatorname{tr} A)^2 < 4 \det A$; and for a triangular matrix (zeros above or below the diagonal) the eigenvalues are just the diagonal entries. The two-tub matrix is triangular, so its eigenvalues are -1 and -2.

This shortcut only works for 2×2 matrices. For bigger ones, a computer finds the eigenvalues numerically.

9 The suspension in numbers, and settling time

slides 38–39

9.1 The suspension in numbers

For the suspension, $\det(\lambda I - A) = \lambda(\lambda + \frac{c}{m}) + \frac{k}{m} = \lambda^2 + \frac{c}{m}\lambda + \frac{k}{m}$. With no damper ($c = 0$) this is $\lambda^2 + k/m = 0$, so $\lambda = \pm j\sqrt{k/m}$: the car would bounce forever at $\omega = \sqrt{k/m}$ rad/s. This is called the *natural frequency*: a stiffer spring or a lighter car bounces faster. Adding damping moves the eigenvalues to the left, so the bouncing dies out.

Keep the same car, $m = 1$ and $k = 4$, and change only the shock absorber:

shock absorber	$\operatorname{tr} A,\ \det A$	eigenvalues	behaviour
good ($c = 2$)	$-2,\ 4$	$-1 \pm j1.73$	a couple of bounces, then still
worn ($c = 1$)	$-1,\ 4$	$-0.5 \pm j1.94$	more bounces, then still
none ($c = 0$)	$0,\ 4$	$\pm j2$	bounces forever (marginal)
pumps energy in ($c = -1$)	$+1,\ 4$	$+0.5 \pm j1.94$	bounces get bigger (unstable)

For $c = 2$ the quadratic formula gives $\lambda = (-2 \pm \sqrt{4-16})/2 = -1 \pm j\sqrt{3} \approx -1 \pm j1.73$. A negative c would be a "shock absorber" that pushes energy into the motion instead of removing it; physically odd, but the same kind of growing oscillation happens when a microphone picks up its own speaker.

9.2 Which mode matters most

Each mode fades like $e^{\sigma_i t}$. The one with the smallest $|\sigma|$, that is, the eigenvalue closest to the imaginary axis, fades last, so it decides how long the system takes to settle. The *settling time* t_s (the time after which the response stays within 2% of its final value) is approximately

$$t_s \approx \frac{4}{|\sigma_{\text{slowest}}|}.$$

The 4 comes from $e^{-4} \approx 0.018$: one time constant is $T = 1/|\sigma|$, and after 4 time constants ($4/|\sigma|$ seconds) $e^{\sigma t}$ has fallen to about 2%. This is the $4T$ rule from Section 3. For eigenvalues -1 and

-10, the e^{-10t} mode is gone after about 0.4 s but the e^{-t} mode takes about 4 s, so $t_s \approx 4$ s. Like a group hike, the system moves at the pace of its slowest member.

10 Transfer functions

slides 36–37

State space describes a system from the inside: all the states and how they push on each other. A *transfer function* describes it from the outside: one input goes in, one output comes out, and the transfer function says how the output responds. Both describe the same system, and stability reads the same in both.

10.1 What a transfer function is

Recall from Section 1.3 that $\frac{d}{dt}e^{st} = s\,e^{st}$. So if a signal behaves like e^{st}, taking its rate of change is the same as multiplying it by s. That is the whole trick: replace every "rate of change of" by "times s", and the differential equations become ordinary algebra. You can think of s as a rate. It plays the same role as λ did for eigenvalues, and like λ it may be a complex number $\sigma + j\omega$.

The formal tool that does this is the *Laplace transform*. It turns the input $u(t)$ into a function $U(s)$, the output $y(t)$ into $Y(s)$, and (starting from rest) each time derivative into a multiplication by s. The transfer function is the ratio of the two,

$$G(s) = \frac{Y(s)}{U(s)},$$

the output divided by the input. For the single bathtub, $\dot{x} = ax + bu$ becomes $sX = aX + bU$, so $X = \frac{b}{s-a}\,U$ and $G(s) = \frac{b}{s-a}$. (Appendix A has a little more on the transform itself.)

10.2 From state space to a transfer function

Do the same with $\dot{\mathbf{x}} = A\mathbf{x} + Bu$. Writing $X(s)$ for the transformed state vector,

$$sX(s) = AX(s) + BU(s) \;\Rightarrow\; (sI - A)X(s) = BU(s) \;\Rightarrow\; X(s) = (sI - A)^{-1}BU(s),$$

where $(sI - A)^{-1}$ is the matrix inverse (Section 1.8). With the output $Y = CX + DU$,

$$G(s) = C(sI - A)^{-1}B + D.$$

The row C says which output you are looking at. The same A and B give a different transfer function for each output you could measure.

10.3 Poles are eigenvalues

The inverse formula divides by the determinant:

$$(sI - A)^{-1} = \frac{\text{adj}(sI - A)}{\det(sI - A)},$$

where the *adjugate* adj is the matrix in the inverse formula before the division (for 2×2: swap the diagonal entries and change the sign of the other two). So every transfer function has the form

$$G(s) = \frac{\text{a polynomial in } s}{\det(sI - A)},$$

with the characteristic polynomial underneath. The *poles* are the values of s that make the denominator zero. Setting $\det(sI - A) = 0$ is the characteristic equation of Section 6.5 with s written in place of λ, so *the poles of the transfer function are the eigenvalues of A*. The stability

rule is the same in both languages: stable if every pole (every eigenvalue) has negative real part, unstable if any has positive real part.

One caution. Sometimes a factor of the numerator cancels a factor of the denominator. The cancelled eigenvalue is still part of the system, but it no longer shows up as a pole of that transfer function. The eigenvalues of A are always the complete list; a single transfer function may show only some of them. When you check stability both ways the answers should agree, and if a pole seems to be missing, look for a cancellation.

10.4 Zeros

The values of s that make the numerator zero are the *zeros*. At a zero, an input that behaves like e^{st} produces no matching e^{st} term in the output: the system blocks that particular rate. Zeros do not decide whether the system on its own is stable (only the poles do), but they shape the response and they matter a great deal once feedback is added.

- When an output is fed back to the input through a gain, the closed-loop poles start at the open-loop poles and, as the gain is increased, move towards the zeros (Section 17). A zero is where a pole ends up under high gain. (If there are more poles than zeros, the extra poles head off to infinity.)
- A zero in the right half-plane ($\operatorname{Re} s > 0$) is a warning sign. High-gain feedback pulls a pole towards it, that is, into the unstable region. Such systems also tend to start moving the wrong way before going the right way.
- A zero close to a pole nearly cancels it. If that pole is unstable, the output hardly "sees" it, so feedback from that output can do little about it.

Different outputs of the same system have the same poles but, in general, different zeros. So the choice of what you measure changes what feedback can achieve, even though the system itself is unchanged.

10.5 The two tubs

For the two tubs (Section 4), $A = \begin{bmatrix} -1 & 0 \\ 1 & -2 \end{bmatrix}$ and $B = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$. Then

$$sI - A = \begin{bmatrix} s+1 & 0 \\ -1 & s+2 \end{bmatrix}, \qquad \det(sI - A) = (s+1)(s+2), \qquad \operatorname{adj}(sI - A) = \begin{bmatrix} s+2 & 0 \\ 1 & s+1 \end{bmatrix},$$

so $\operatorname{adj}(sI - A)\, B = [s+2,\ 1]^T$ and $D = 0$. Now pick an output.

- **Level of tub 2**, $C = [0 \ \ 1]$: $G(s) = \dfrac{1}{(s+1)(s+2)}$. Poles -1 and -2, the eigenvalues. No zeros.
- **Level of tub 1**, $C = [1 \ \ 0]$: $G(s) = \dfrac{s+2}{(s+1)(s+2)} = \dfrac{1}{s+1}$. The zero at -2 cancels the pole at -2. This makes sense physically: tub 2 never affects tub 1, so tub 2's rate cannot appear in tub 1's level. The eigenvalue -2 is still there, just invisible from this output.
- **Total water**, $C = [1 \ \ 1]$: $G(s) = \dfrac{(s+2)+1}{(s+1)(s+2)} = \dfrac{s+3}{(s+1)(s+2)}$. Same poles, and now a zero at -3.

Three outputs, one set of poles, three different sets of zeros. All three are stable, as the eigenvalues already said.

11 Summary of Part 1 *slides 40*

To analyse any linear system:

1. Count the energy stores. That is the number of states n.
2. Choose one state per store (or the variable and its derivatives).
3. Write $\dot{x}_i$ for each state using only states and inputs.
4. Collect the numbers into $\dot{\mathbf{x}} = A\mathbf{x} + Bu$.
5. Solve $\det(\lambda I - A) = 0$ for the eigenvalues. For 2×2 you can use the trace and determinant test instead.
6. Read the eigenvalues: any positive real part means unstable; all negative means stable; imaginary parts mean oscillation; the slowest real part gives the settling time.
7. For each output of interest, the transfer function $C(sI - A)^{-1}B + D$ has the eigenvalues as poles; its zeros depend on which output you measure.

Common mistakes.

- Leaving a derivative on the right-hand side of a state equation.
- Saying a system is stable because most of its eigenvalues are negative. One positive real part is enough to make it unstable.
- Treating marginally stable as stable.
- Forgetting the minus signs in $\lambda I - A$.
- Using the trace/determinant test on a matrix bigger than 2×2.

Part 2 applies all of this to the acrobot. It adds two new ideas: what to do when the equations are not linear, and how feedback changes the eigenvalues.

Part II

The acrobot

12 The acrobot and the three tasks *slides 41–43*

The acrobot is two links in a vertical plane, joined end to end. The top joint (the shoulder) is a free pivot with no motor. The middle joint (the elbow) has the only motor. With two joints but only one motor, it is called *underactuated*.

A good picture is a gymnast on a high bar. The hands grip the bar but can apply almost no turning effort there, so the grip acts like a free pivot and all the effort comes from bending at the hips. Even so, a gymnast can swing higher and higher and end in a handstand above the bar. The acrobot does the same with a motor at the "hips".

There are three tasks: bring the robot to rest hanging down, balance it upright, and swing it up from hanging to upright. The first two stay close to one position, so the linear tools from Part 1 work once we know how to get a linear model of a nonlinear robot. The third covers half a circle and needs a different idea, based on energy.

13 Nonlinear systems and linear models *slides 44–48*

Equilibria

Keys hanging from a hook and a pencil balanced on its tip are both at rest. Tap the keys and they swing back to hanging; tap the pencil and it falls. A pendulum can do both: hanging and balanced upright are the same object and the same equation at two different resting points.

For a nonlinear system $\dot{\mathbf{x}} = f(\mathbf{x}, u)$, an equilibrium is a pair $(\mathbf{x}^*, u^*)$ with

$$f(\mathbf{x}^*, u^*) = 0.$$

For a mechanical system with no input this means all speeds are zero and the gravity torque at every joint is zero. For the pendulum $\ddot{\theta} = -\frac{g}{\ell}\sin\theta$, the equilibria are where $\sin\theta = 0$: $\theta^* = 0$ (hanging) and $\theta^* = \pi$ (upright).

Why linear models work near an equilibrium

The pendulum in a grandfather clock only swings a little, and the size of the swing hardly changes the tick rate. This is because for small angles $\sin\theta \approx \theta$, and the equation becomes linear. The approximation is good for small angles and gets worse as the angle grows:

θ (rad)	$\sin\theta$	error
0.1	0.0998	0.2%
0.5	0.479	4%
1.0	0.841	16%

Close to an equilibrium, any smooth nonlinear system behaves approximately like a linear one, in the same way that the Earth is round but a street looks flat. Far from the equilibrium the approximation fails. This will matter later when we ask how far from upright the balancing controller can cope.

Two more tools: partial derivatives and straight-line approximations

A *partial derivative* is a rate of change when a function depends on several variables and you change only one of them. If $f(x_1, x_2) = x_1^2 x_2$, then $\partial f/\partial x_1$ ("partial f by partial x one") treats x_2 as a fixed number and differentiates with respect to x_1, giving $2x_1x_2$; and $\partial f/\partial x_2 = x_1^2$. It answers the question: if I nudge x_1 a little and leave x_2 alone, how much does f change?

A *straight-line (first-order) approximation*: near a point x^*, a smooth function looks like its tangent line,

$$f(x^* + \delta x) \approx f(x^*) + f'(x^*)\,\delta x,$$

where δx ("delta x") is a small change and f' is the derivative. For example, near $\theta = 0$, $\sin\theta \approx 0 + \cos(0)\,\theta = \theta$. Near $\theta = \pi$, $\sin(\pi + \delta) \approx 0 + \cos(\pi)\,\delta = -\delta$. This is the first term of the Taylor series. With several variables, each variable gets its own partial derivative term.

Linearisation

Write $\delta\mathbf{x} = \mathbf{x} - \mathbf{x}^*$ and $\delta u = u - u^*$ and keep the first-order terms of the Taylor series. Since $f(\mathbf{x}^*, u^*) = 0$,

$$\delta\dot{\mathbf{x}} = A\,\delta\mathbf{x} + B\,\delta u, \qquad A = \left.\frac{\partial f}{\partial \mathbf{x}}\right|_*, \qquad B = \left.\frac{\partial f}{\partial u}\right|_*.$$

A is the Jacobian of f, with entries $A_{ij} = \partial f_i/\partial x_j$ evaluated at the equilibrium.

Lyapunov's indirect method connects the linear model back to the real system. If all eigenvalues of A have negative real part, the equilibrium $\mathbf{x}^*$ of the nonlinear system is asymptotically stable: a state that starts close enough returns to it. If any has positive real part, the equilibrium is unstable. If some lie on the imaginary axis and none to the right, the linear model cannot decide. So the method only gives an answer when no eigenvalue is on the imaginary axis.

Pendulum

The pendulum equation is $\ddot{\theta} = -\frac{g}{\ell}\sin\theta$. The states are $x_1 = \theta$ and $x_2 = \dot{\theta}$, so

$$\dot{\mathbf{x}} = f(\mathbf{x}) = \begin{bmatrix} \dot{\theta} \\ -\frac{g}{\ell}\sin\theta \end{bmatrix}, \qquad \frac{\partial f}{\partial \mathbf{x}} = \begin{bmatrix} 0 & 1 \\ -\frac{g}{\ell}\cos\theta & 0 \end{bmatrix}.$$

In words, A says how f changes when you nudge each state. Entry (i, j) is how much row i of f moves per unit nudge of state j. The top row is $\dot{\theta}$: it moves by 1 per unit of $\dot{\theta}$ and not at all with θ. The bottom row does not depend on $\dot{\theta}$, and its change with θ depends on where you are.

Hanging ($\theta^* = 0$): there $\sin\theta \approx \theta$, so the bottom row is about $-\frac{g}{\ell}\theta$ and

$$A = \begin{bmatrix} 0 & 1 \\ -g/\ell & 0 \end{bmatrix}.$$

Upright ($\theta^* = \pi$): there $\sin\theta \approx -(\theta - \pi)$, so the bottom row is about $+\frac{g}{\ell}(\theta - \pi)$ and

$$A = \begin{bmatrix} 0 & 1 \\ +g/\ell & 0 \end{bmatrix}.$$

From these two matrices, $\det(\lambda I - A) = \lambda^2 + g/\ell$ when hanging and $\lambda^2 - g/\ell$ when upright. Hanging gives $\lambda = \pm j\sqrt{g/\ell}$: without friction it swings forever (marginal). Upright gives $\lambda = \pm\sqrt{g/\ell}$: one eigenvalue is positive, so it falls. Hanging, gravity pulls it back; upright, gravity pushes it away. The only change between the two cases is the sign of one entry. The acrobot works the same way with four states, and all the gravity entries of its A change sign in the same way.

Linearising numerically

When f is complicated, the Jacobian can be computed by perturbing one state at a time:

$$A_{:,j} \approx \frac{f(\mathbf{x}^* + \varepsilon\mathbf{e}_j, u^*) - f(\mathbf{x}^* - \varepsilon\mathbf{e}_j, u^*)}{2\varepsilon}.$$

Here $\mathbf{e}_j$ is the vector with a 1 in entry j and zeros elsewhere, and ε is a small number (for example 10^{-6}). In words: nudge state j up a little and down a little, see how much f changes, and divide by the size of the nudge. That gives column j of A. Doing the same with the input gives B. This only needs a way to evaluate $f(\mathbf{x}, u)$, so it works with any simulation model, and it is a good check on a Jacobian worked out by hand.

14 Feedback, controllability and LQR

slides 49–62

Why not plan the pushes ahead?

One idea is *open loop* control: work out all the motor pushes beforehand, then just play them back. At upright this fails. The fast unstable mode of the acrobot grows like $e^{7.19\,t}$, so an error doubles about every 0.1 s. In one second it grows by a factor of $e^{7.19} \approx 1300$, so an error of 0.001 rad, too small to see, becomes more than 1 rad. No plan made in advance can be that exact. It is like walking with your eyes shut. You have to keep looking and correcting, and that is feedback.

Feedback

Adjusting a shower: you feel the water and turn the tap towards hot or cold, more if it is further off. Balancing a broom on your palm is the same idea, faster. In both cases you measure the state and set the input from it.

With state feedback $u = -K\mathbf{x}$,

$$\dot{\mathbf{x}} = A\mathbf{x} + B(-K\mathbf{x}) = (A - BK)\mathbf{x}.$$

The closed loop has a new matrix $A - BK$, and choosing K moves its eigenvalues. You cannot change gravity, but you can change how the system behaves.

One knob: proportional control. Take the single tub again, with x the distance from the level you want: $\dot{x} = -x + u$. Push harder the further off it is, $u = -k\,x$. Then

$$\dot{x} = -(1 + k)\,x, \qquad \lambda = -1 - k.$$

With $k = 0$ the eigenvalue is -1; $k = 2$ moves it to -3 and $k = 5$ to -6. Turning k up slides the eigenvalue left, so the level settles faster. This is *proportional* (P) control. It holds for this one-state tub. With more states, or with a motor limit, too much gain can backfire.

P, I and D. The most common controller in practice is PID:

$$u = k_p\,e + k_i \int e\,dt + k_d\,\dot{e},$$

where e is the error, $e =$ where you want to be $-$ where you are.

- P pushes harder the further off you are. It acts like a spring.
- D pushes against the speed of the error, so it overshoots less. It acts like a damper.
- I keeps adding up the leftover error and pushes until it is gone. It acts like a memory. It is needed when something keeps pushing back, like a car going uphill: P alone settles a bit short of the target speed, and I sees the leftover error and keeps adding until it is fixed.

PID is used in cruise control, ovens, drones and motor speed control.

PD is adding a spring and a damper. For the suspension, $m\ddot{p} = -kp - c\dot{p} + u$. With $u = -k_p p - k_d \dot{p}$, the state $[p,\ \dot{p}]$ and $K = [k_p \ \ k_d]$,

$$A - BK = \begin{bmatrix} 0 & 1 \\ -\frac{k+k_p}{m} & -\frac{c+k_d}{m} \end{bmatrix}.$$

This is the suspension with a spring of stiffness $k + k_p$ and a damper of strength $c + k_d$. So k_p gives a stiffer spring and k_d a thicker damper: tuning a PD controller is picking new shocks for the car, in software.

Why not PID for everything? PID looks at one error and drives one motor. The acrobot has four states and one motor, so it is not clear which error to look at. The states also affect each other, so turning one knob upsets the others. It is simpler to use every state at once, $u = -K\mathbf{x}$ with $K = [k_1\ k_2\ k_3\ k_4]$. PD is the special case where $\mathbf{x}$ is position and speed and $K = [k_p \ \ k_d]$.

Pole placement. For a normalised upright pendulum (the broom), $A = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$, $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, with eigenvalues ± 1. With $K = [k_1\ k_2]$,

$$A - BK = \begin{bmatrix} 0 & 1 \\ 1 - k_1 & -k_2 \end{bmatrix}, \qquad \operatorname{tr} = -k_2, \quad \det = k_1 - 1.$$

It is stable for $k_2 > 0$, $k_1 > 1$. To place both eigenvalues at -2, match $\lambda^2 + 4\lambda + 4$: $k_2 = 4$, $k_1 = 5$. Here k_1 reacts to how far the broom leans and k_2 to how fast it is falling, which is essentially PD control.

The same steps work for a cart on ice, with no friction and the input as its acceleration: $\ddot{x} = u$. With state $[x,\ \dot{x}]$, $A = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix}$ and $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, so

$$A - BK = \begin{bmatrix} 0 & 1 \\ -k_1 & -k_2 \end{bmatrix}, \qquad \det(\lambda I - (A - BK)) = \lambda^2 + k_2\lambda + k_1.$$

To put both eigenvalues at -2, match $(\lambda + 2)^2 = \lambda^2 + 4\lambda + 4$: $k_1 = 4$, $k_2 = 4$, so $K = [4 \quad 4]$. This is the example on the slides. The broom needs $k_1 = 5$ instead of 4 because one unit of the gain goes into cancelling gravity's push.

So where do you put them? Further left settles faster, but it needs a bigger K and so bigger pushes. Real motors have a limit, and a big K also amplifies sensor noise. And with four eigenvalues, it is not obvious where all four should go. Pole placement says how to get the eigenvalues somewhere. It does not say where that should be. The next subsection turns specifications into a region, and LQR (below) is another answer.

From what you want to where the poles go

slides 58

Pole placement says how to put the eigenvalues anywhere. Specifications say where. For a pair of complex poles $\sigma \pm j\omega$ (and no zeros) the two usual specifications translate directly:

- *Settling time.* The response dies like $e^{\sigma t}$, so it is within about 2% after $4/|\sigma|$. "Settle within T" means $\sigma \le -4/T$: the poles must lie left of a vertical line.
- *Overshoot.* It depends only on the damping ratio $\zeta = |\sigma|/\sqrt{\sigma^2 + \omega^2}$, which is the cosine of the angle between the pole and the negative real axis: overshoot $= e^{-\pi\zeta/\sqrt{1-\zeta^2}}$. For at most 5% you need $\zeta \ge 0.69$, an angle of at most about 46°, so the poles must lie inside a wedge around the negative real axis.

Together these give a region of the plane, and any poles inside it meet both specifications. For the cart on ice, "settle within 2 s with at most 5% overshoot" allows, for example, $-3 \pm 2j$ ($4/3 \approx 1.3$ s, $\zeta = 3/\sqrt{13} \approx 0.83$). Matching $(\lambda + 3)^2 + 4 = \lambda^2 + 6\lambda + 13$ gives $k_1 = 13$, $k_2 = 6$. (A repeated real pole is a special case: at -2 twice the response is $(1 + 2t)e^{-2t}$, which takes about 2.9 s to get within 2%, longer than $4/2$.)

With more than two poles these rules are a guide. If the other poles are several times further left than the slowest pair, that pair dominates and the rules work well; otherwise check by simulation. LQR does not take pole locations directly: you choose Q and R, look at the closed-loop poles, and adjust (more weight in Q generally pushes them left) until they are inside the region.

Zeros behave differently. State feedback $u = -K\mathbf{x}$ moves the poles but leaves the zeros of the transfer function from the input to a measured output where they were, because the zeros are set by where the input enters (B) and what is measured (C). A controller with its own dynamics can add zeros: a PD controller adds one, which tends to make the response faster but can increase overshoot. A zero in the right half-plane cannot be removed by feedback and limits how fast the closed loop can safely be made.

Controllability

A car cannot move sideways directly, but you can still parallel park by combining forward and reverse moves with steering; the car's own motion carries you sideways. A shopping trolley with a jammed wheel is different: some motions are impossible whatever you do.

The two tubs give a simple example. The tap only fills tub 1, so you cannot pour into tub 2. You can still set tub 2's level, because tub 1 drains into it. Here $B = [1,\ 0]^T$ is the direct push (into tub 1), and $AB = [-1,\ 1]^T$ is where the system carries that push next (into tub 2). Together they cover both states.

In state space, B gives the directions the input can push directly, AB the directions the dynamics carry that push into next, then A^2B, and so on. If these together span every direction, any state can be reached. The system is controllable if

$$\operatorname{rank}\begin{bmatrix} B & AB & A^2B & \cdots & A^{n-1}B \end{bmatrix} = n,$$

and in that case the eigenvalues of $A - BK$ can be placed anywhere (complex ones in conjugate pairs). The test does not say how much input this takes, and the input needed can be very large. The *rank* of a matrix is the number of genuinely different directions its columns point in. For a square matrix, full rank is the same as a non-zero determinant. So for two states the test is simply $\det[B \;\; AB] \neq 0$. For the broom, $[B\ AB] = \begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$ has rank 2. For the two tubs, $\det[B \;\; AB] = \det \begin{bmatrix} 1 & -1 \\ 0 & 1 \end{bmatrix} = 1 \neq 0$, so they are controllable.

For the acrobot this is the question of whether one motor at the elbow is enough. Like the car, it cannot push the shoulder directly, but the coupling through the dynamics may still make every state reachable.

LQR

Choosing four eigenvalue locations by hand is awkward. LQR instead asks you to state what you care about. Think of drifting off the middle of your lane. You can yank the wheel and be back in the middle quickly but jerkily, or ease it back smoothly and take longer. Neither is wrong; it is a trade-off between error (how far off you are) and effort (how hard you steer). LQR minimises

$$J = \int_0^\infty \left(\mathbf{x}^T Q\,\mathbf{x} + u^T R\,u\right) dt.$$

The integral sign $\int_0^\infty \ldots\, dt$ means "add up over all time from now on". Q is usually a diagonal matrix, written $\operatorname{diag}(q_1, \ldots, q_n)$, whose entries say how much each state's error matters; $\mathbf{x}^T Q \mathbf{x} = q_1 x_1^2 + \cdots + q_n x_n^2$. R is a number here (one motor) saying how much the input effort costs. LQR finds the K with the lowest J. Large Q penalises state error: the driver who yanks the wheel, fast but jerky. Large R penalises input effort: the driver who eases it back, smooth but slow. Q and R say which of the two drivers you want.

LQR with one state. Take $\dot{x} = a\,x + u$ and $J = \int_0^\infty (q\,x^2 + r\,u^2)\,dt$. The best controller moves the eigenvalue to

$$\lambda_{\text{new}} = -\sqrt{a^2 + q/r},$$

where q/r is how much you dislike being off divided by how much you dislike pushing. If $a > 0$ the system falls on its own. When effort is expensive (q/r small), $\lambda_{\text{new}} \approx -a$: LQR flips the unstable eigenvalue to its mirror image, the cheapest fix there is. When effort is cheap (q/r large), it pushes the eigenvalue further left.

The full version. You pick Q (which states matter) and R (how much pushing costs). The optimal gain is $K = R^{-1}B^{T}S$, where S solves the algebraic Riccati equation

$$A^{T}S + SA - SBR^{-1}B^{T}S + Q = 0.$$

You do not solve this by hand; it is solved numerically by a standard routine that returns K and S from A, B, Q and R. The quantity $\mathbf{x}^{T}S\mathbf{x}$ is the score still to come from where you are: the total cost J that the LQR controller will run up starting from state $\mathbf{x}$, as predicted by the linear model. We will use it to decide when to switch controllers. LQR is used in balancing robots, autopilots and satellites.

So far: every controller just picks K

P and PID, pole placement and LQR are all ways of choosing K in $u = -K\mathbf{x}$. The gain K moves the eigenvalues of $A - BK$. Controllable means you can put them anywhere you like, and LQR picks where by trading error against effort. It is one idea throughout: look at the state, push back, and decide how hard.

15 The acrobot model

slides 63–65

Coordinates

q_1 is the shoulder angle measured from straight down, and q_2 is the elbow angle measured relative to link 1. The parameters are the link masses m_1, m_2, lengths ℓ_1, ℓ_2, distances from each joint to the link's centre of mass ℓ_{c1}, ℓ_{c2}, and the link inertias about their own pivots I_1, I_2. The state is $\mathbf{x} = [q_1, q_2, \dot{q}_1, \dot{q}_2]^{T}$ and the input is the elbow torque τ (in the general notation of Part 1, $u = \tau$). The motor can only deliver a limited torque, $|\tau| \le \tau_{\max}$. These conventions follow Spong (1995) and Tedrake's *Underactuated Robotics*.

Equations of motion

A *torque* is the turning effect of a force, measured in newton-metres (N m); for rotation it plays the role that force plays for straight-line motion. The equations of a linked robot are usually derived with Lagrange's method, which starts from the kinetic energy T and potential energy U and produces one equation of motion for each joint. The details are not needed here. The result for the acrobot has the standard robot form

$$M(\mathbf{q})\ddot{\mathbf{q}} + C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} = \boldsymbol{\tau}_g(\mathbf{q}) + B_q\tau, \qquad B_q = \begin{bmatrix} 0 \\ 1 \end{bmatrix},$$

with

$$M = \begin{bmatrix} I_1 + I_2 + m_2\ell_1^2 + 2m_2\ell_1\ell_{c2}\cos q_2 & I_2 + m_2\ell_1\ell_{c2}\cos q_2 \\ I_2 + m_2\ell_1\ell_{c2}\cos q_2 & I_2 \end{bmatrix},$$

$$C = \begin{bmatrix} -2m_2\ell_1\ell_{c2}\sin q_2\,\dot{q}_2 & -m_2\ell_1\ell_{c2}\sin q_2\,\dot{q}_2 \\ m_2\ell_1\ell_{c2}\sin q_2\,\dot{q}_1 & 0 \end{bmatrix},$$

$$\boldsymbol{\tau}_g = \begin{bmatrix} -m_1 g\ell_{c1}\sin q_1 - m_2 g\big(\ell_1\sin q_1 + \ell_{c2}\sin(q_1 + q_2)\big) \\ -m_2 g\ell_{c2}\sin(q_1 + q_2) \end{bmatrix}.$$

M is the inertia matrix (the equivalent of m in the suspension), C contains the Coriolis and centrifugal terms, and $\boldsymbol{\tau}_g$ is the vector of gravity torques (playing the role of the spring; do not confuse it with the motor torque τ). $B_q = [0,\ 1]^{T}$ says the motor torque acts only at the elbow.

M^{-1} is the inverse matrix of M (Section 1.8). Multiplying by it is how we "divide by the mass" when the mass is a matrix. Solving for the accelerations gives the state-space form

$$\dot{\mathbf{x}} = f(\mathbf{x}, \tau) = \begin{bmatrix} \dot{\mathbf{q}} \\ M(\mathbf{q})^{-1}\big(\boldsymbol{\tau}_g(\mathbf{q}) + B_q\tau - C(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}}\big) \end{bmatrix},$$

with the same structure as the suspension: the top half says the angles change at the angular speeds and the bottom half is Newton's law for rotation. Because of the sines and cosines it is nonlinear.

The teaching robot

The numbers in these notes are for a teaching robot with two identical uniform links: $m_1 = m_2 = 1$ kg, $\ell_1 = \ell_2 = 1$ m, $\ell_{c1} = \ell_{c2} = 0.5$ m (the centre of each link), inertias $\frac{1}{12}$ kg m^2 about each centre of mass (so $I_1 = I_2 = \frac{1}{12} + 1 \cdot 0.5^2 = \frac{1}{3}$ kg m^2 about the pivots, by the parallel-axis theorem), $g = 9.81$ m/s^2, and a motor torque limit $\tau_{\max} = 10$ N m. The inertias in M are about the pivots; if your data give them about the centre of mass, convert them the same way, $I = I_c + m\ell_c^2$. This is deliberately a different robot from the one in the project. The method carries over step by step, but the numbers do not: every number below has to be recomputed for your own robot.

A real robot also has joint friction. It is left out here to keep the equations short; adding it moves the downright eigenvalues slightly into the left half-plane.

Equilibria

With $\dot{\mathbf{q}} = 0$ and $\tau = 0$, gravity has to balance, so $\sin q_1 = 0$ and $\sin(q_1 + q_2) = 0$. This gives four equilibria (π rad is 180°, so $q_1 = \pi$ points straight up): $(0, 0)$ with both links down (downright), $(\pi, 0)$ with both links up (upright), and two mixed ones $(0, \pi)$ and (π, π) with one link folded back. All four count as equilibria and all four need to be analysed. The worked examples below use the first two, $\mathbf{x}^* = [q_1^*, 0, 0, 0]^T$ with $q_1^* = 0$ or π, and $\tau^* = 0$. The mixed ones are linearised in exactly the same way, with one caution: the block formula for $\partial\boldsymbol{\tau}_g/\partial\mathbf{q}$ in the next section assumes $q_2 = 0$. At the mixed equilibria $q_2 = \pi$, so $\cos q_1$ and $\cos(q_1 + q_2)$ have opposite signs and M changes too ($\cos q_2 = -1$). Redo the derivative there rather than reusing the formula.

16 Linear models and stability of the acrobot *slides 66–68*

At an equilibrium $\dot{\mathbf{q}} = 0$, and every term of $C\dot{\mathbf{q}}$ is quadratic in the velocities, so its derivative vanishes there. Also $\boldsymbol{\tau}_g + B_q\tau^* = 0$ there, so the terms from differentiating $M(\mathbf{q})^{-1}$ vanish too. Linearising f leaves

$$A = \begin{bmatrix} 0 & I \\ M^{-1}\dfrac{\partial\boldsymbol{\tau}_g}{\partial\mathbf{q}} & 0 \end{bmatrix}, \qquad B = \begin{bmatrix} 0 \\ M^{-1}B_q \end{bmatrix},$$

with M evaluated at $q_2 = 0$. Differentiating $\boldsymbol{\tau}_g$ and substituting $\cos q_1 = \cos(q_1 + q_2) = \pm 1$:

$$\frac{\partial\boldsymbol{\tau}_g}{\partial\mathbf{q}} = \mp g \begin{bmatrix} m_1\ell_{c1} + m_2\ell_1 + m_2\ell_{c2} & m_2\ell_{c2} \\ m_2\ell_{c2} & m_2\ell_{c2} \end{bmatrix},$$

with the minus sign at downright and the plus sign at upright. As with the pendulum, moving from down to up flips the sign of the gravity block.

With the teaching-robot parameters, $M = \begin{bmatrix} 2.667 & 0.833 \\ 0.833 & 0.333 \end{bmatrix}$ and $B = [0,\ 0,\ -4.29,\ 13.7]^T$ at both equilibria. At downright

$$A = \begin{bmatrix} 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \\ -12.61 & 12.61 & 0 & 0 \\ 16.82 & -46.25 & 0 & 0 \end{bmatrix},$$

and at upright the lower-left block changes sign. Because A has the block form $\begin{bmatrix} 0 & I \\ G & 0 \end{bmatrix}$, its eigenvalues satisfy $\lambda^2 =$ eigenvalues of G, so they always come in $\pm$ pairs.

	eigenvalues	stability
downright	$\pm j2.68,\ \pm j7.19$	stable, not asymptotically
upright	$\pm 2.68,\ \pm 7.19$	unstable

Downright. There are two oscillation modes, analogous to bounce and pitch. The slow mode (2.68 rad/s, period 2.34 s) has eigenvector with $q_2 \approx 0.431\, q_1$: both links swing together. The fast mode (7.19 rad/s, period 0.874 s) has $q_2 \approx -3.10\, q_1$: the links scissor against each other. With no friction neither mode decays. All four eigenvalues are on the imaginary axis, which is exactly the case where Lyapunov's indirect method (Section 13) cannot decide, so the linear model alone does not settle stability here. Energy does. Without friction the total energy is conserved, and it has its minimum at hanging still. A small nudge gives the robot a little energy above that minimum, and it can never gain more, so it can never move far from hanging. The downright equilibrium is therefore stable, but not asymptotically stable: it keeps swinging and never settles.

Upright. The fast unstable mode grows like $e^{7.19\,t}$, so errors double every $\ln 2/7.19 \approx 0.096$ s. (The slow unstable mode, $e^{2.68\,t}$, doubles every 0.26 s.) An error of 0.05 rad grows to about 1 rad in $\ln 20/7.19 \approx 0.4$ s, so the robot falls over in about half a second, which means the balancing controller has to act quickly.

Controllability. The controllability matrix has rank 4 at both equilibria, so the elbow motor can in principle place all four eigenvalues. A large condition number of this matrix (the ratio of its largest to smallest "stretch") is a first sign that some directions are hard to control; for the teaching robot it is about 6.4×10^3, which is large. To see which direction is the hard one, measure how strongly the torque pushes on each mode (the size of B along that mode). For the 7.19 modes it is 1.97; for the 2.68 modes it is only 0.047. The slow mode is roughly 40 times harder to push than the fast one (the exact ratio depends on how the states are scaled, e.g. radians against radians per second, so treat it as a rough guide), so the robot is controllable, but only barely through that mode. This one fact explains most of what happens in the next section.

Slide 68 shows the nonlinear model nudged by 0.05 rad from each equilibrium with no control. At downright the two modes mix and never die out; at upright the error becomes large within about half a second. This agrees with the eigenvalues.

Transfer functions of the linearised robot

Section 10 turns the linear model at an equilibrium into a transfer function for each angle we might measure: $C = [1\ 0\ 0\ 0]$ gives the transfer function from τ to the shoulder angle q_1, and $C = [0\ 1\ 0\ 0]$ the one from τ to the elbow angle q_2 ($D = 0$ in both, since a torque cannot change

an angle instantly). Both have the denominator $\det(sI - A)$, so both have the same four poles: the eigenvalues in the table above. Their numerators differ, so their zeros differ.

The block form keeps the algebra short. Write G for the lower-left block $M^{-1}\partial\boldsymbol{\tau}_g/\partial\mathbf{q}$ (entries g_{ij}) and $\mathbf{b} = M^{-1}B_q = [b_1,\ b_2]^T$ for the lower half of B. The top half of the state equation says the speeds are the rates of change of the angles, so after the transform the speeds are s times the angles, and the bottom half becomes $s^2\mathbf{Q}(s) = G\,\mathbf{Q}(s) + \mathbf{b}\,T(s)$, where $\mathbf{Q}(s)$ is the transformed angle vector and $T(s)$ the transformed torque. Hence $\mathbf{Q}(s) = (s^2I - G)^{-1}\mathbf{b}\,T(s)$, and using the 2×2 inverse,

$$\frac{Q_1(s)}{T(s)} = \frac{b_1 s^2 + (g_{12}b_2 - g_{22}b_1)}{\det(s^2I - G)}, \qquad \frac{Q_2(s)}{T(s)} = \frac{b_2 s^2 + (g_{21}b_1 - g_{11}b_2)}{\det(s^2I - G)},$$

with $\det(s^2I - G) = \det(sI - A)$. Two things are worth noticing.

- Only even powers of s appear (this comes from leaving out friction). So if z is a pole or a zero, so is $-z$.
- Each transfer function has two zeros, at $s^2 = -(\text{constant term})/(\text{coefficient of } s^2)$. If that ratio is negative, the zeros are an imaginary pair $\pm j\omega_z$. If it is positive, they are a real pair $\pm z$, and one of them is in the right half-plane. Which case you get depends on the robot and the equilibrium, and it can differ between q_1 and q_2.

For the teaching robot at upright,

$$\frac{Q_1(s)}{T(s)} = \frac{-4.29\,s^2 + 25.2}{s^4 - 58.9\,s^2 + 371}, \qquad \frac{Q_2(s)}{T(s)} = \frac{13.7\,s^2 - 101}{s^4 - 58.9\,s^2 + 371},$$

with zeros at ± 2.43 for q_1 and ± 2.71 for q_2. Both have a zero in the right half-plane, and both of those zeros sit close to the unstable pole at $+2.68$; the q_2 zero almost cancels it. At downright the transfer functions are

$$\frac{Q_1(s)}{T(s)} = \frac{-4.29\,s^2 - 25.2}{s^4 + 58.9\,s^2 + 371}, \qquad \frac{Q_2(s)}{T(s)} = \frac{13.7\,s^2 + 101}{s^4 + 58.9\,s^2 + 371},$$

with imaginary zeros at $\pm j2.43$ and $\pm j2.71$. The q_2 zeros are again close to the slow-mode poles $\pm j2.68$.

17 Acrobot controllers *slides 69–79*

All the linear controllers below use $\tau = -K\,\delta\mathbf{x}$, the general $u = -K\mathbf{x}$ of Section 14 with the state measured from the equilibrium.

Downright

The goal is to bring a swinging acrobot to rest. LQR on the downright model with $Q = \text{diag}(10, 1, 10, 1)$ and $R = 0.1$ gives

$$K = [-26.5,\ -4.84,\ -3.07,\ 3.54], \qquad \text{closed-loop eigenvalues } -60.1,\ -1.37,\ -0.139 \pm j2.59.$$

The closed loop is stable. The slowest closed-loop eigenvalues are $-0.139 \pm j2.59$, giving a settling time of about $4/0.139 \approx 29$ s. In simulation, a 0.1 rad nudge in q_1 takes about 33 s to settle (peak torque 2.7 N m), and a 0.1 rad nudge in q_2 about 24 s. That slow pair is the slow "swing together" mode, still oscillating at almost its open-loop 2.68 rad/s and only just pulled off the imaginary axis. This is the weakly controllable mode from the previous section: the elbow is a poor lever on the whole-body swing, so the controller can only damp it gently. Controllable does not mean easy.

Upright

LQR on the upright model, with the error measured from $(\pi, 0, 0, 0)$ and $Q = \mathrm{diag}(10, 10, 1, 1)$, $R = 1$, gives

$$K = [-310,\ -78.7,\ -114,\ -33.6], \qquad \text{closed-loop eigenvalues } -17.1,\ -4.02,\ -2.68 \pm j0.04.$$

All four eigenvalues are now in the left half-plane. Look at the last pair, though: $-2.68 \pm j0.04$ is almost exactly the open-loop unstable eigenvalue $+2.68$ flipped to the other side of the axis. The controller has turned the slow mode from growing to decaying and done little more, because the elbow can barely push on it. The weak mode shows up again here. Started 0.02 rad off in the shoulder and -0.02 rad in the elbow, with the 10 N m torque limit, the elbow swings out to about 0.41 rad to catch the fall, everything settles in a few seconds (3.4 s for a 0.02 rad shoulder offset on its own), and the peak torque is about 4.6 N m, inside $\tau_{\max} = 10$ N m. A 0.1 rad offset in either joint is already too much and the robot falls. The motor does hit its 10 N m limit, but the real cause is the weak slow mode explained below: it falls even with no limit. When implementing it, the angle error must be measured from π and wrapped into $(-\pi, \pi]$.

Can you stabilise it by measuring only one joint angle? *slides 69*

The LQR above is *full state feedback*: $\tau = -K\,\delta\mathbf{x}$ needs all four states, both angles and both speeds. A natural question is whether less will do. Suppose only one joint angle y is measured ($y = \delta q_1$ or $y = \delta q_2$), and the torque is computed from it alone. This is *output feedback*. The two simplest versions are

- a static gain, $\tau = -k\,y$;
- PD on that one angle, $\tau = -k_p\,y - k_d\,\dot{y}$, which also uses the rate of change of the same angle (found by differentiating the measurement), but nothing from the other joint.

How to check. Write the transfer function from τ to the measured angle as $G(s) = n(s)/d(s)$, with $d(s) = \det(sI - A)$. With $\tau = -k\,y$ we have $Y = G\,T = -kG\,Y$, so the closed loop has poles where $1 + kG(s) = 0$, that is, at the roots of

$$d(s) + k\,n(s) = 0, \qquad \text{and for PD} \qquad d(s) + (k_p + k_d s)\,n(s) = 0.$$

In state space the same thing reads: $\tau = -kC\mathbf{x}$ gives the closed-loop matrix $A - kBC$, whose eigenvalues are these roots. The question is whether *any* choice of gains, positive or negative, puts all the roots in the left half-plane. Two tools answer it.

- **Root locus.** Plot the roots as the gain is swept from 0 to large values (and to large negative values, since the sign is yours to choose). At zero gain they sit at the open-loop poles. As the gain grows, the closed-loop poles go from the open-loop poles towards the zeros of $n(s)$ (or off to infinity, for the ones left over when there are fewer zeros than poles). If the paths of the unstable poles stay in the right half-plane for every gain, no gain works. A right-half-plane zero is fatal for high gain, because a root ends up close to it. A zero close to an unstable pole is also bad: the measurement hardly sees that mode, so the pole barely moves unless the gain is very large.

- **Coefficient check.** If all the roots of a polynomial are in the left half-plane, then all its coefficients are non-zero and have the same sign. (The reverse is not true beyond second order; the Routh–Hurwitz test gives the full condition, and a computer can simply find the roots.) This often rules a controller out at a glance. For the frictionless robot, $d(s)$ and $n(s)$ contain only even powers of s (previous section), so $d(s) + k\,n(s)$ has no odd powers at all. Its roots come in $\pm$ pairs, and a static gain on one angle can never make them all

stable. A PD term $k_d s\, n(s)$ supplies the odd powers; whether their coefficients can all get the right sign depends on the signs in $n(s)$, that is, on where its zeros are.

What the answer depends on. So the zeros decide it. The same robot can be stabilisable from one angle and not from the other, because the two transfer functions have the same poles but different zeros. This is a statement about simple controllers fed from one angle. A more elaborate controller can estimate the missing states from the one measurement (an *observer*) and then act like full state feedback, provided that angle carries information about every state; even then a right-half-plane zero limits how well it can work. When you answer "can it be stabilised from one angle?", say which kind of feedback you mean.

For the teaching robot at upright, neither angle works on its own, with either kind of feedback.

- **Static gain.** $d(s) + k\, n(s)$ has only even powers of s, so its roots are symmetric about the imaginary axis and can never all be in the left half-plane. From q_1 the best you can do is marginal: for k between about -14.7 and -14.6 all four roots sit on the imaginary axis. (For example $k = -14$ gives $-1.36 \pm j1.55$ and $+1.36 \pm j1.55$.) As $|k|$ grows, two roots go to the zeros ± 2.43, so one stays stuck near $+2.43$. From q_2 it is not even marginal: a real root stays between $+2.68$ and $+2.71$ for every k, held there by the right-half-plane zero at $+2.71$. The coefficients show the same thing: the s^2 coefficient $-58.9 + 13.7k$ is positive only for $k > 4.29$, while the constant $371 - 101k$ is positive only for $k < 3.67$.

- **PD on the same joint.** With $n(s) = as^2 + b$, the closed-loop polynomial is $s^4 + ak_d s^3 + (-58.9 + ak_p)s^2 + bk_d s + (371 + bk_p)$. The s^3 and s coefficients have the same sign only if a and b do. Here they have opposite signs (-4.29 and $+25.2$ for q_1, 13.7 and -101 for q_2), which is exactly the statement that the zeros $\pm\sqrt{-b/a}$ are real, with one in the right half-plane. So no k_p, k_d can work. A scan of about 233,000 gain pairs agrees: none is stable. The best from q_1 still has a root at $+0.72$; the best from q_2 has one at $+2.71$, trapped by the zero.

So the unstable slow mode at $+2.68$ is the problem again. From either angle, a right-half-plane zero sits right next to it, and from q_2 the near cancellation makes that mode almost invisible. In principle an observer-based controller could still stabilise it from one angle, but these simple ones cannot.

Region of attraction

When you knock a glass, you can save it if it has only just started tipping, but not once it is already falling off the table. The upright LQR is similar. Tested on the nonlinear model with the 10 N m torque limit, it recovers from an initial shoulder offset of 0.02 rad (about 1 degree) but not from 0.03 rad. That is very little.

The obvious suspects are not to blame. With no torque limit at all, 0.03 rad still falls. Raising R all the way to 1000, or trying other Q matrices, leaves the boundary at 0.02 rad. The weak link is the slow 2.68 rad/s mode. The elbow is about 40 times worse at pushing on it than on the fast mode, so to correct it at all the controller has to swing the links a long way: from a 0.02 rad start the shoulder swings out to -0.19 rad and the elbow to 0.68 rad before things settle. Start a little further out and these swings carry the robot beyond the range where the linear model holds. The robot is controllable, but controllable does not mean easy. This is a property of this particular robot: on another robot, including the project one, the torque limit can well be the deciding factor, so check both with and without the limit. The direction you start in matters a lot: with $q_2 = -4q_1$, which keeps the centre of mass over the pivot, it recovers from $[q_1, q_2]$ offsets of $[0.05,\ -0.2]$ rad with a peak torque of only 1.2 N m.

The set of initial states from which a controller succeeds is its region of attraction. The LQR was designed from a linear model, which is only accurate near upright (see the $\sin\theta$ table in Section 13), so its region of attraction is small. Starting from hanging down, something else must first bring the acrobot close to upright.

Torque limits *slides 72*

A real motor can only give so much torque. If the controller asks for τ_{cmd}, the motor delivers

$$\tau = \begin{cases} \tau_{\text{cmd}} & \text{if } |\tau_{\text{cmd}}| \le \tau_{\max}, \\ \pm\tau_{\max} \text{ (with the sign of } \tau_{\text{cmd}}) & \text{otherwise.} \end{cases}$$

This is called *saturation*. While the motor is saturated the loop is no longer linear: the torque stops growing with the error, so twice the error no longer gives twice the correction. The eigenvalues of $A - BK$ describe the loop only while the requested torque stays inside the limit.

Why the region of attraction shrinks. Near upright, gravity pulls the robot over harder the further it leans, but the motor can push back with at most $\tau_{\max}$. Close to upright the requested torque is small and the loop behaves as designed. Further out the controller asks for more than the motor has and gets only $\tau_{\max}$, and beyond some point gravity wins whatever the controller does. A controller that works without the limit can fail with it, so always simulate with the limit included.

How to measure "how far from the equilibrium it still works". There is no formula; it is measured on the full nonlinear model with the torque limit in place.

1. Decide what counts as success, for example that both angle errors are small and stay small after a fixed time.
2. Choose a direction to start in: a shoulder offset only, an elbow offset only, an initial speed, or a combination.
3. Start close to the equilibrium and move the starting point further and further away until the controller fails. Halving the gap between the last success and the first failure pins the boundary down quickly.
4. Repeat for other directions, or scan a grid of two starting values (say the q_1 and q_2 offsets) and mark each point as success or failure. The successful area is your estimate of the region of attraction.

Report the boundary together with the success criterion and the value of $\tau_{\max}$ used.

R trades effort for speed. In LQR, R is the price of torque. A larger R gives smaller gains: a gentler, slower controller that asks for less torque and so saturates later. A smaller R gives a faster, more aggressive controller that hits the limit sooner. With a torque limit, the most aggressive design is not necessarily the one with the largest region of attraction, so it is worth repeating the measurement for a few values of R (and of Q) and comparing. For the teaching robot with $\tau_{\max} = 10$ N m, the boundary in shoulder offset is 0.02 rad for every R tried from 1 to 1000. A larger R lowers the shoulder gain only from 310 to 227, and the region does not grow. Here the boundary is set by the weak mode, not by the torque; that is specific to this robot, and on another robot the torque limit can be what sets it. Hanging down the picture is the opposite: the downright LQR with the same limit recovers from shoulder offsets of 3.1 rad, almost the whole swing range.

Swing-up: the energy idea

A child on a swing goes higher without being pushed by moving their legs in time with the swing, adding a little energy each cycle, and stops pumping once high enough. When you push someone on a swing you push while they are moving away from you: pushing in the direction of motion adds energy, pushing against it removes energy.

Going from hanging to upright is a change of about π rad, well outside the range of any linear model, so instead of tracking angles we track the total energy E, kinetic plus potential:

$$E(\mathbf{x}) = \tfrac{1}{2}\dot{\mathbf{q}}^T M(\mathbf{q})\dot{\mathbf{q}} - m_1 g \ell_{c1}\cos q_1 - m_2 g\big(\ell_1 \cos q_1 + \ell_{c2}\cos(q_1+q_2)\big).$$

At rest upright, $E_{\text{up}} = g(m_1\ell_{c1} + m_2\ell_1 + m_2\ell_{c2}) = 19.62$ J. At rest hanging it is -19.62 J, so about 39.2 J has to be added. Gravity does not change the total energy; only the motor does, and its power is torque times joint speed:

$$\dot{E} = \dot{q}_2\,\tau.$$

Pushing the elbow in the direction it is already moving increases the energy.

Swing-up: partial feedback linearisation

The motor acts at the elbow, but $\ddot{q}_2$ depends on both joints through M. Partial feedback linearisation chooses the torque so that the elbow follows any desired acceleration v. It does this by solving the equations of motion for the torque that gives that acceleration. With $\boldsymbol{\phi} = \boldsymbol{\tau}_g - C\dot{\mathbf{q}}$, the two rows of the equations of motion are (with m_{ij} the entries of $M(\mathbf{q})$)

$$m_{11}\ddot{q}_1 + m_{12}\ddot{q}_2 = \phi_1, \qquad m_{12}\ddot{q}_1 + m_{22}\ddot{q}_2 = \phi_2 + \tau.$$

From the first row, $\ddot{q}_1 = (\phi_1 - m_{12}\ddot{q}_2)/m_{11}$. Substituting into the second and setting $\ddot{q}_2 = v$:

$$\tau = \Big(m_{22} - \frac{m_{12}^2}{m_{11}}\Big)v + \frac{m_{12}}{m_{11}}\phi_1 - \phi_2.$$

This only works while that torque is within $\pm\tau_{\max}$; when the motor saturates, the elbow no longer follows v.

Swing-up: the pumping law

Spong's energy-pumping law, in the form used by Tedrake, is

$$\bar{u} = (E_{\text{up}} - E)\,\dot{q}_1, \qquad v = -k_1 q_2 - k_2\dot{q}_2 + k_3\bar{u}.$$

The PD terms $-k_1 q_2 - k_2\dot{q}_2$ keep the elbow roughly straight. The term $k_3\bar{u}$ bends the elbow in time with the shoulder swing ($\dot{q}_1$), scaled by how much energy is still missing, so the pumping stops by itself when $E = E_{\text{up}}$. In the child-on-a-swing picture: keep the body straight, move the legs in time with the swing, pump hard when low and stop when high enough. The gains used here are $k_1 = 50$, $k_2 = 5$, $k_3 = 2$.

This is a nonlinear controller, so it is not analysed with eigenvalues. Its progress is measured by whether the energy reaches E_{up}. The pumping term is a rule of thumb (Spong's), with no general guarantee: $\dot{E} = \dot{q}_2\tau$, while the law multiplies by the shoulder speed $\dot{q}_1$, so it is not built to make the energy rise at every instant. The thing to watch is $|E - E_{\text{up}}|$ shrinking on the energy plot. Changing the gains (even the sign of k_3) changes how the energy builds up, so judge each choice from that plot.

Switching

In a relay race the first runner covers the long distance and hands the baton to the second runner in a marked zone. The acrobot works the same way: the swing-up controller brings it near the top and the upright LQR takes over. The handover happens when

$$\mathbf{x}^T S \mathbf{x} < 10,$$

where S is the Riccati solution from the upright LQR and $\mathbf{x}$ is the error from upright. This is the LQR's score from here. A small value means the predicted cost of balancing from here is small, i.e. the state is close enough for the linear controller. The threshold was tuned in simulation so that the handover lands inside the region of attraction. Since that region is so small for this robot, the threshold has to be small too: in tests (with $k_3 = 3$), values from 2 to 25 all caught it, smaller ones on a later swing, while 40 or more handed over states the LQR could not hold. If the robot is knocked out of that region, control goes back to swing-up.

Simulation result

Slide 77 shows the full simulation from $q_1 = 0.1$ rad. Measured from E_{up}, the energy starts at about -39.2 J and barely changes for the first 10 s or so, does most of its rise in a few seconds (roughly 12 to 18 s), and then creeps up to zero. The handover to LQR happens at 32.84 s, with the energy 0.18 J short of E_{up}, and the acrobot stays balanced: by the end of the 43 s run the error is zero to four decimal places. The peak torque during swing-up is 10 N m, that is, the motor reaches $\tau_{\max} = 10$ N m, but it is saturated only about 3% of the time. After the switch the LQR needs at most 7.9 N m. The elbow stays within 2.1 rad during the swing-up. If q_1 ends at $-\pi$ in the plot, that is the same physical position as $+\pi$. The run starts 0.1 rad off on purpose. From exactly hanging still ($\mathbf{x} = 0$), q_2, $\dot{q}_2$ and $\dot{q}_1$ are all zero, so $v = 0$; the torque that gives $v = 0$ is also zero there, and the robot never moves. In the project, where the swing-up starts from hanging straight down, first give it a small kick (a short torque pulse) or any tiny non-zero starting state.

The swing-up is sensitive to k_3. With k_1, k_2 and the switching rule as above (75 s runs):

- $k_3 = 0.5$ never hands over. The pumping is too weak: after 40 s the energy is still 8.3 J short, and the motor never saturates.
- $k_3 = 1$ is gentler but slower: handover at 57.45 s, and it holds.
- $k_3 = 2$ (the choice above): handover at 32.84 s, and it holds.
- $k_3 = 3$ is faster (handover at 27.75 s) and holds, but the elbow wraps round to 5.04 rad, more than half a turn, and the motor is saturated 10% of the time.
- $k_3 = 4$ fails. The motor is saturated 79% of the time, so partial feedback linearisation can no longer make the elbow follow v; the elbow spins and the state never reaches the handover set.

In between, $k_3 = 3.5$ switched at 17 s with 10.9 J too much energy. It arrived far too fast, outside the LQR's region of attraction, and fell. More pumping is not simply faster: past a point it saturates the motor or overshoots the energy.

Explaining why a swing-up works or not *slides 78*

A swing-up either reaches upright and stays there or it does not. Either way, the useful part is explaining why. The scheme has several stages that can each go wrong (pumping energy in, keeping the elbow under control, arriving in the right configuration, handing over, catching), and each one leaves evidence in the simulation. Things worth plotting and reporting:

- **Energy against time**, with a line at E_{up}. Does it rise steadily, stall below E_{up}, or overshoot? A stall can mean the pumping signal $\bar{u}$ is small (it is proportional to $\dot{q}_1$) or that the torque is saturated. Reaching E_{up} is necessary but not enough: the right energy can come with the wrong configuration, for example the robot whirling round, or the links far from straight.
- **Torque against time**, with lines at $\pm\tau_{\max}$. How much of the time is the motor saturated? While it is, partial feedback linearisation no longer holds, the elbow does not follow the requested v, and the reasoning behind the pumping law stops applying.
- **The joint angles and speeds**, especially the elbow angle. Does the PD part keep it bounded?
- **The state at handover**: both angles, both speeds, and $\mathbf{x}^T S\mathbf{x}$ compared with the threshold. Compare that state with the region of attraction you measured for the upright controller with the torque limit. A handover outside it explains a failed catch. A handover well inside it, followed by a failure, points somewhere else.
- **The torque just after handover.** Does the LQR immediately ask for more than $\tau_{\max}$?
- **Sensitivity.** Change one thing at a time (k_1, k_2, k_3, the switching threshold, the starting state, $\tau_{\max}$) and record what happens: time to handover, peak torque, success or failure. A small table of this is strong evidence. If success depends on one lucky setting, say so.

A good explanation links cause and effect with numbers, along the lines of "the catch fails because at handover $\mathbf{x}^T S\mathbf{x}$ is this, the state is outside the measured region of attraction in this direction, and the motor is saturated for this much of the last swing". Every claim should point to a plot or a number.

Which controller when

All of these are ways of picking u.

- **P, PI, PID**: one thing to control, easy to tune. Cruise control, ovens, motors.
- **Pole placement**: small systems where you pick the speed. Simple lab rigs.
- **LQR**: many states, trading speed against effort. Balancing robots, autopilots.
- **Model predictive control (MPC)**: plans the pushes ahead using the model, applies the first one, then re-plans at every step, and keeps to limits such as $\tau_{\max}$. Self-driving cars, factories.
- **Feedback linearisation**: uses the model to work out a torque that cancels the nonlinear terms, so what is left behaves linearly. Robot arms, drones. The partial version above does this for the elbow only.
- **Energy-based**: far from rest, pumping energy in. Our swing-up.
- **Switching**: glue that hands over when close enough. Our acrobot.

The controllers built on a linear model only work near the resting point. The others can work further out and bring the system close enough for them.

18 Summary of Part 2 *slides 80*

For a nonlinear system:

1. Write the model $\dot{\mathbf{x}} = f(\mathbf{x}, u)$, one state per energy store.
2. Find the equilibria from $f(\mathbf{x}^*, u^*) = 0$.
3. Linearise to get A and B at each equilibrium (by hand, or numerically).
4. Use Part 1 on A: eigenvalues and stability. Check it against the poles of the transfer functions to each measured output, and note their zeros.
5. Check controllability.
6. Design a local controller (for example LQR) and check the eigenvalues of $A - BK$. If only one output is measured, use the zeros and the root locus to see whether simple output feedback can work.
7. Test it on the full nonlinear model, with the torque limit, to see how far from the equilibrium it still works.
8. If the task starts far away, use a nonlinear method (here, energy) to get close, then switch. Whether or not it works, explain why with evidence.

More common mistakes: trusting a linear model far from its equilibrium; forgetting to measure the angle error from π at upright; assuming controllable means easy; simulating without the torque limit.

Exercises, glossary and references

19 Exercises

1. A tub has floor area $A_t = 0.5$ m^2 and drain resistance $R = 2$ s/m^2. Find a and b in $\dot{h} = ah + bq_{\text{in}}$. What is the time constant, and roughly how long does it take to settle?
2. Multiply out $\begin{bmatrix} 2 & -1 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 4 \\ 1 \end{bmatrix}$.
3. Compute $\det \begin{bmatrix} 3 & 1 \\ 4 & 2 \end{bmatrix}$ and $\det \begin{bmatrix} 2 & 4 \\ 1 & 2 \end{bmatrix}$. For the one with zero determinant, find a non-zero $\mathbf{v}$ with $M\mathbf{v} = 0$.
4. Write a state-space model for $2\ddot{y} + 6\dot{y} + 4y = u$.
5. Find the eigenvalues and eigenvectors of your A from Exercise 4. Is it stable? Does it oscillate? What is the settling time?
6. Without solving, classify as stable, marginal or unstable: (a) $\begin{bmatrix} -2 & 1 \\ 0 & -5 \end{bmatrix}$, (b) $\begin{bmatrix} 1 & 2 \\ 3 & 0 \end{bmatrix}$, (c) $\begin{bmatrix} 0 & 1 \\ -9 & 0 \end{bmatrix}$.
7. Find the roots of $\lambda^2 + 4\lambda + 13 = 0$. What are σ and ω? Describe the motion.
8. (Part 2) Linearise $\ddot{\theta} = -\frac{g}{\ell}\sin\theta - b\dot{\theta}$ (a pendulum with friction $b > 0$) at $\theta^* = 0$ and $\theta^* = \pi$. What does b change?

9. (Part 2) For $A = \begin{bmatrix} 0 & 1 \\ 2 & -1 \end{bmatrix}$ and $B = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, find K that puts both eigenvalues of $A - BK$ at -3.

10. (Part 2) In a simulation of the swing-up, change the pumping gain k_3. What happens to the swing-up time, to the elbow angle and to how often the motor saturates? Does the LQR still catch it?

11. (Think) A ball is thrown straight up. How many states does it need, and what are they? Why is height alone not enough?

12. (Think) In the bathtub model $\dot{x} = ax + bu$, why does x appear on the right-hand side? What would it mean physically if a were positive?

13. (Think) Two tubs: tub 1 drains into tub 2. Why is the bottom-left entry of A non-zero but the top-right entry zero?

14. (Think) Why does a pole at -10 hardly matter next to a pole at -1 when you ask how long the system takes to settle?

15. (Think) A friend says "$\det(\lambda I - A) = 0$ means the system is broken." What does it actually mean, and why is it exactly what we want?

16. (Think) Why can't you balance a broom with your eyes shut, even if you planned every hand movement perfectly in advance?

17. (Think) Two identical tubs are filled by one tap that pours equally into both. Can you set their levels to different values? What does this say about controllability?

18. (Think) If pushing is made very expensive in LQR (large R), where does an unstable eigenvalue end up, and why is that the "cheapest" fix?

19. (Think) The motor hits its torque limit. Why does that shrink the range of starting angles the upright controller can recover from?

20. (Think) In a swing-up, the energy rises and then flattens out below E_{up} while the torque is stuck at its limit. What is going on, and what would you try?

Worked answers

1. $a = -1/(A_t R) = -1/(0.5 \cdot 2) = -1$ s^{-1} and $b = 1/A_t = 2$ m^{-2}. Time constant $T = -1/a = 1$ s, so it settles in about $4T = 4$ s.

2. Row 1: $2 \cdot 4 + (-1) \cdot 1 = 7$. Row 2: $0 \cdot 4 + 3 \cdot 1 = 3$. Answer $[7,\ 3]^T$.

3. $3 \cdot 2 - 1 \cdot 4 = 2$. $2 \cdot 2 - 4 \cdot 1 = 0$. For the second, $2v_1 + 4v_2 = 0$ gives $v_1 = -2v_2$, so $\mathbf{v} = [-2,\ 1]^T$ (check: $2(-2) + 4(1) = 0$ and $1(-2) + 2(1) = 0$).

4. Divide by 2: $\ddot{y} + 3\dot{y} + 2y = \frac{1}{2}u$. With $x_1 = y$, $x_2 = \dot{y}$: $A = \begin{bmatrix} 0 & 1 \\ -2 & -3 \end{bmatrix}$, $B = \begin{bmatrix} 0 \\ \frac{1}{2} \end{bmatrix}$, $C = [1\ \ 0]$.

5. This is the matrix from Section 6.6: $\lambda = -1, -2$ with eigenvectors $[1,\ -1]^T$ and $[1,\ -2]^T$. Both negative: stable. Both real: no oscillation. Slowest $|\sigma| = 1$, so $t_s \approx 4$ s.

6. (a) Triangular, eigenvalues -2 and -5: stable. (b) $\det = 0 - 6 = -6 < 0$, so one eigenvalue is positive: unstable. (c) $\text{tr} = 0$, $\det = 9$: $\lambda^2 + 9 = 0$, $\lambda = \pm j3$: marginal (oscillates forever at 3 rad/s).

7. $\lambda = (-4 \pm \sqrt{16 - 52})/2 = (-4 \pm j6)/2 = -2 \pm j3$. So $\sigma = -2$, $\omega = 3$ rad/s. The motion oscillates at 3 rad/s inside an envelope e^{-2t}; it is stable and settles in about $4/2 = 2$ s.

8. $A = \begin{bmatrix} 0 & 1 \\ -\frac{g}{\ell}\cos\theta^* & -b \end{bmatrix}$. At 0: $\mathrm{tr} = -b < 0$, $\det = g/\ell > 0$, so it is now stable instead of marginal. At π: $\det = -g/\ell < 0$, still unstable. Friction helps the hanging case, not the upright one.

9. $A - BK = \begin{bmatrix} 0 & 1 \\ 2 - k_1 & -1 - k_2 \end{bmatrix}$, with $\det(\lambda I - (A - BK)) = \lambda^2 + (1 + k_2)\lambda + (k_1 - 2)$. We want $(\lambda + 3)^2 = \lambda^2 + 6\lambda + 9$, so $k_2 = 5$ and $k_1 = 11$.

10. With $k_1 = 50$, $k_2 = 5$ and handover at $\mathbf{x}^T S \mathbf{x} < 10$: $k_3 = 0.5$ pumps too weakly and never hands over in 75 s; $k_3 = 1$ is gentler but slower (handover at 57.45 s); $k_3 = 2$ hands over at 32.84 s; $k_3 = 3$ is faster (27.75 s) and still caught, but the elbow wraps round to 5.04 rad and the motor is saturated 10% of the time; $k_3 = 4$ fails, because the motor is saturated 79% of the time, the elbow no longer follows v, and the state never reaches the handover set. In every case also check the torque against $\tau_{\max} = 10$ N m.

11. Two: height and vertical speed. Two balls at the same height, one rising and one falling, will be in different places a moment later, so height alone cannot predict the future.

12. The drain's outflow depends on how much water is already there, so the level's own value sets how fast it changes. A positive a would mean more water makes the level rise faster, a runaway, which a real drain cannot do.

13. Row 2 says how tub 2's level changes; tub 1 pours into it, so h_1 appears there. Row 1 says how tub 1 changes; water does not flow uphill from tub 2, so h_2 does not appear.

14. The part from -10 dies like e^{-10t}, gone in about 0.4 s, while the -1 part takes about 4 s. You end up waiting for the slow one.

15. It means $\lambda I - A$ flattens the plane, so some non-zero arrow gets sent to zero. That arrow satisfies $A\mathbf{v} = \lambda\mathbf{v}$: it is a special direction. Nothing is broken; this is how we find them.

16. Upright, small errors grow exponentially, and real errors (a gust, a slightly wrong starting angle) are never exactly what you planned for. Without watching and correcting, a tiny mistake becomes a fall. That is why we need feedback.

17. No. Whatever you do, both tubs receive the same inflow and drain the same way, so their levels stay equal (or their difference just dies away on its own). The input can never push the difference, so the system is not controllable.

18. To its mirror image on the left: $+a$ becomes $-a$. That is the smallest move that makes it stable, so it needs the least pushing.

19. The LQR was designed assuming it can push as hard as it likes. Further from the top it asks for more torque than the motor has, so it pushes less than it needs to and the robot falls before it can recover.

20. The motor is too weak, or the pumping law is fighting itself, so each swing adds too little energy (or loses some). Check the energy and torque plots, try a different k_3, or let it swing for longer before handing over.

Glossary of symbols

Symbol	Meaning
t	time (s)
$\dot{x}$, $\ddot{x}$	rate of change of x; rate of change of $\dot{x}$
$x(0)$	value of x at $t = 0$
e	the number $2.718\ldots$
$\mathbf{x}$	state vector (one entry per energy store)
n	number of states
u	input (what we control)
y	output (what we measure)
A	$n \times n$ matrix: what the system does by itself
B	$n \times 1$: how the input pushes the states
C	$1 \times n$: what we measure
D	direct input-to-output term, usually 0
A_{ij}	entry of A in row i, column j
I	identity matrix
$\det M$	determinant of M ($ps - qr$ for 2×2)
$\operatorname{tr} A$	trace of A: sum of the diagonal entries
λ	eigenvalue
$\mathbf{v}$	eigenvector
c_i	amount of eigenvector i in the starting state
j	$\sqrt{-1}$
σ, ω	real and imaginary parts of an eigenvalue
$\operatorname{Re}\lambda$	real part of λ
t_s	settling time (within 2%)
T	time constant (s)
$\delta\mathbf{x}$	small change from a resting state
$f(\mathbf{x}, u)$	right-hand side of a nonlinear model
$\mathbf{x}^*$, u^*	state and input at an equilibrium
$\partial f/\partial \mathbf{x}$	partial derivatives (the Jacobian)
K	feedback gains, $u = -K\mathbf{x}$
Q, R, S, J	LQR weights, Riccati solution, cost
s	transfer-function variable (a "rate")
$G(s)$	transfer function, output over input
pole, zero	root of the denominator, of the numerator of $G(s)$
k, k_p, k_d	output-feedback gains (static, PD)
M^{-1}	inverse of the matrix M
q_1, q_2	acrobot shoulder and elbow angles
τ, $\tau_{\max}$	acrobot elbow torque (the input, $u = \tau$), motor torque limit
M, C, $\boldsymbol{\tau}_g$	acrobot inertia matrix, Coriolis and centrifugal terms, gravity torques
m_i, ℓ_i, ℓ_{ci}, I_i	link mass, length, joint-to-centre distance, inertia
E, E_{up}	total energy, energy at rest upright
v, $\bar{u}$, k_1, k_2, k_3	swing-up: desired elbow acceleration, pumping signal, gains
Δt	small time step in a simulation
h, q_{in}, A_t, R	tub level, tap flow, floor area, drain resistance
p, m, k, c	suspension position, mass, spring stiffness, damping
a_0, a_1, b	constant coefficients of a general second-order equation

A More detail on transfer functions

Section 10 gives everything the course needs. This appendix adds two details for readers who want them.

Where s comes from. The Laplace transform turns a function of time $x(t)$ into a function $X(s)$ of a new variable s. Its most useful property is that a time derivative becomes a multiplication:

the transform of $\dot{x}$ is $sX(s) - x(0)$. Starting from rest ($x(0) = 0$) this is just $sX(s)$, which is why every $\dot{\mathbf{x}}$ in the state equation turned into $sX(s)$. A non-zero starting state adds an extra term $(sI - A)^{-1}\mathbf{x}(0)$ to the response, with the same denominator $\det(sI - A)$, so it does not change the poles.

Check with the suspension. $sI - A = \begin{bmatrix} s & -1 \\ k/m & s + c/m \end{bmatrix}$, with determinant $s(s+c/m)+k/m = s^2 + \frac{c}{m}s + \frac{k}{m}$. Its adjugate is $\begin{bmatrix} s + c/m & 1 \\ -k/m & s \end{bmatrix}$. Then C adj $B = [1 \;\; 0] \begin{bmatrix} s + c/m & 1 \\ -k/m & s \end{bmatrix} \begin{bmatrix} 0 \\ 1/m \end{bmatrix} = 1/m$, so

$$G(s) = \frac{1/m}{s^2 + \frac{c}{m}s + \frac{k}{m}} = \frac{1}{ms^2 + cs + k},$$

which is exactly what you get by Laplace transforming $m\ddot{p} + c\dot{p} + kp = u$ directly (replace each derivative by a factor s). The poles are the suspension eigenvalues from Section 9, and there are no zeros.

Further reading and references

- K. Ogata, *Modern Control Engineering*, 5th ed., Prentice Hall, 2010. Chapters on state-space analysis.
- G. F. Franklin, J. D. Powell and A. Emami-Naeini, *Feedback Control of Dynamic Systems*, Pearson. Chapter on state-space design.
- M. W. Spong, "The swing up control problem for the Acrobot," *IEEE Control Systems Magazine*, vol. 15, no. 1, pp. 49–55, 1995.
- R. Tedrake, *Underactuated Robotics*, MIT course notes, chapter "Acrobots, Cart-Poles, and Quadrotors." `https://underactuated.mit.edu`
- G. Strang, *Introduction to Linear Algebra*, Wellesley-Cambridge Press. Chapters on determinants and eigenvalues, if you want more on the matrix side.